

TLXML: TASK-LEVEL EXPLANATION OF META-LEARNING VIA INFLUENCE FUNCTIONS

A PREPRINT

Yoshihiro Mitsuka^{1 *}

Shadan Golestan²

Zahin Sufiyan³

Shotaro Miwa⁴

Osmar R. Zaiane^{2,3}

¹Information Technology R&D Center, Mitsubishi Electric Corporation

²Alberta Machine Intelligence Institute

³Department of Computing Science, University of Alberta

⁴Advanced Technology R&D Center, Mitsubishi Electric Corporation

January 27, 2026

ABSTRACT

Meta-learning enables models to rapidly adapt to new tasks by leveraging prior experience, but its adaptation mechanisms remain opaque, especially regarding how past training tasks influence future predictions. We introduce TLXML (Task-Level eXplanation of Meta-Learning), a novel framework that extends influence functions to meta-learning settings, enabling task-level explanations of adaptation and inference. By reformulating influence functions for bi-level optimization, TLXML quantifies the contribution of each meta-training task to the adapted model’s behaviour. To ensure scalability, we propose a Gauss-Newton-based approximation that significantly reduces computational complexity from $O(pq^2)$ to $O(pq)$, where p and q denote model and meta parameters, respectively. Results demonstrate that TLXML effectively ranks training tasks by their influence on downstream performance, offering concise and intuitive explanations aligned with user-level abstraction. This work provides a critical step toward interpretable and trustworthy meta-learning systems.

1 Introduction

Meta-learning, or “learning to learn”, equips models with the ability to rapidly adapt to unseen tasks, addressing limitations in generalization caused by data scarcity during training (Song and Jeong, 2024; Li et al., 2018; Shu et al., 2021; Lu et al., 2021) and distribution shifts in deployment environments (Mann et al., 2021; Mouli et al., 2024; Lin et al., 2020). Despite its growing success and fast adaptation scenarios, meta-learning remains largely opaque. Current approaches offer limited insight into which training tasks influence the adaptation process and final predictions, creating significant barriers for transparency, trust, and safe deployment in real-world applications (Zhang et al., 2020; Khattar et al., 2023; Wen et al., 2022; Yao et al., 2023). Consequently, robust explanation methods are required to enhance transparency and ensure the safe and reliable operation of autonomous systems.

Most explanation methods in machine learning focus on local interpretability, aiming to understand model behaviour around individual input examples. While such methods can be adopted to explain meta-learners’ post-adaptation performances (the box on the right in Figure 1), they fall short of capturing the unique characteristics of meta-learning. Notably, the final model behaviour in meta-learning is highly sensitive not only to the test-time support set but also to the collection of prior training tasks, as highlighted by Agarwal et al. (2021). Furthermore, local explanations often require technical expertise to interpret (Adebayo et al., 2022), and may not be helpful for end-users seeking actionable or intuitive insights. In contrast, task-level explanations—those that attribute predictions to previously encountered

*Mitsuka.Yoshihiro@bp.MitsubishiElectric.co.jp

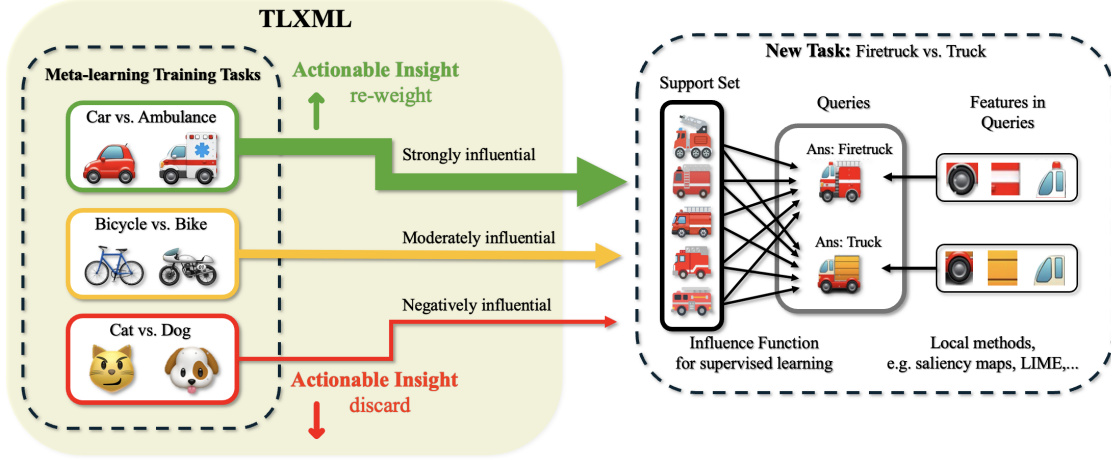


Figure 1: Key insights of TLXML.

tasks—align more naturally with how meta-learning models are trained and how humans reason about prior experience (Figure 1). Understanding how individual tasks contribute to model behaviour is especially important as meta-learning systems increasingly operate over heterogeneous, multi-domain datasets. Task-level insights can improve both the interpretability and safety of such systems by identifying which training contexts are most responsible for certain behaviours.

Influence functions (Hampel, 1974; Cook and Weisberg, 1980) provide a powerful tool for tracing model predictions back to training data, and have been successfully used in standard supervised learning to identify influential data points (Koh and Liang, 2017; Koh et al., 2019), assess robustness (Cohen et al., 2020), detect bias (Han and Tsvetkov, 2020), improve interpretability (Chhabra et al., 2024), and analyze large language models (Grosse et al., 2023). Those methods can also be applied to explain the influence of data points used in adaptation for each task, provided the adaptation algorithm satisfies their underlying assumptions (the box on the right in Figure 1). However, influence functions have not yet been extended to the meta-learning process, where the training examples are tasks, not data points.

In this paper, we propose TLXML (Task-Level eXplanation of Meta-Learning)—a novel framework that applies influence functions to the meta-learning setting. TLXML quantifies the impact of individual meta-training tasks on the model’s adaptation and inference behaviour. We reformulate influence functions to accommodate the bi-level optimization structure inherent in meta-learning algorithms. Our approach allows users to understand how prior tasks shape the learning process in a principled and interpretable way

Contributions. Our main contributions are as follows: 1) Task-Level Explanations for Meta-Learning: We introduce TLXML, a principled method for quantifying the influence of meta-training tasks on adaptation and prediction in meta-learning. It offers concise, interpretable explanations aligned with user abstraction levels. 2) Scalable Influence Computation: We analyze the computational cost of TLXML, showing that the exact method scales poorly with $O(pq^2)$, where p and q are the number of model and meta-parameters. We propose a Gauss-Newton-based approximation that reduces the cost to $O(pq)$, making TLXML feasible for practical use. We also propose generalizing TLXML with the pseudo-inverse Hessian to avoid the instability caused by flat directions around the loss minimization point. 3) Empirical Validation: We empirically demonstrate that TLXML can meaningfully rank meta-training tasks by their influence on adaptation and performance, successfully identifying helpful versus unhelpful training tasks across several benchmarks, and it can be utilized for improving the adaptation of meta-trained models.

2 Related work

Influence functions for supervised learning. The primary focus of existing research is the use of influence functions in supervised learning initiated by Koh and Liang (2017), such as explaining model behavior in terms of training data in various tasks (Barshan et al., 2020; Koh and Liang, 2017; Han et al., 2020), quantifying model uncertainty (Alaa and van der Schaar, 2020), crafting/detecting adversarial training examples (Cohen et al., 2020). Similar to Khanna et al. (2019); Barshan et al. (2020) in supervised learning, this work utilizes the Fisher information metric. It offers a

useful geometric perspective on the method but, more importantly, addresses a computational cost challenge peculiar to meta-learning.

Explainable AI (XAI) for meta-learning. Naturally, XAI methods that are agnostic to the learning process are applicable for explaining inference in meta-learning. Although this area is still in its early stages, several research examples exist (Woźnica and Biecek, 2021; Sijben et al., 2024; Shao et al., 2023). With a motivation close to ours, Woźnica and Biecek (2021) quantified the importance of high-level characteristics of a dataset such as size, number of features and number of classes. However, we emphasize that the goal of meta-learning is to train models that generalize across *tasks*, and understanding the impact of training tasks is crucial for evaluating a model’s adaptability.

Impact of training data. The robustness of machine learning models is another area where the impact of training data is frequently discussed (Khanna et al., 2019; Ribeiro et al., 2016). This topic has also been explored in the context of meta-learning such as creating training-time adversarial attacks via meta-learning (Zügner and Günnemann, 2019; Xu et al., 2021), training robust meta-learning models by exposing models to adversarial attacks during the query step of meta-learning (Goldblum et al., 2020), and data augmentation for enhancing the performance of meta-learning algorithms (Ni et al., 2021). However, influence functions have not been utilized for meta-learning to date.

Machine Unlearning (MU) Removing the influence of training data from a trained model represents a valuable application of influence functions beyond increasing the interpretability of machine learning models. Aiming to handle privacy concerns in supervised learning, Cao and Yang (2015) introduced MU as a method of forgetting a training point without incurring the cost of retraining the model from scratch. Although MU was first proposed outside the scope of machine learning interpretability, subsequent research established close connections. Guo et al. (2020) noticed that the influence function estimate of deleting one data point matches the Newton method update of the weights. Golatkar et al. (2020, 2021) re-derived influence functions from an information-theoretic argument. See Li et al. (2025) for a recent review including those progresses. In this paper, we show that TLXML extends MU to meta-learning and can be used to both block and amplify the influence of training tasks.

3 Preliminaries

Influence functions. Koh and Liang (2017) proposed influence functions for measuring the impact of training data on the outcomes of supervised learning, under the assumption that the trained weights $\hat{\theta}$ minimize the empirical risk:

$$\hat{\theta} = \operatorname{argmin}_{\theta} L(D^{\text{train}}, f_{\theta}) = \operatorname{argmin}_{\theta} \frac{1}{n} \sum_{i=1}^n l(z_i, f_{\theta})$$

where f_{θ} is the model to be trained, $D^{\text{train}} = \{z_i\}_{i=1}^n$ is the training dataset consisting of n pairs of an input x_i and a label y_i , i.e., $z_i = (x_i, y_i)$, and the total loss L is the sum of the losses l of each data point. The influence functions are defined with a perturbation ϵ for each data point z_j :

$$\begin{aligned} \hat{\theta}_{\epsilon,j} &= \operatorname{argmin}_{\theta} L_{\epsilon,j}(D^{\text{train}}, f_{\theta}) \\ &= \operatorname{argmin}_{\theta} \frac{1}{n} \sum_{i=1}^n l(z_i, f_{\theta}) + \frac{\epsilon}{n} l(z_j, f_{\theta}), \end{aligned}$$

which is considered as a shift of the probability that z_j occurs in the data distribution. The influence of the data z_j on the model parameter $\hat{\theta}$ is defined as its increase rate with respect to this perturbation:

$$I^{\text{param}}(j) \stackrel{\text{def}}{=} \left. \frac{d\hat{\theta}_{\epsilon,j}}{d\epsilon} \right|_{\epsilon=0} = -\frac{1}{n} H^{-1} \left. \frac{\partial l(z_j, f_{\theta})}{\partial \theta} \right|_{\theta=\hat{\theta}} \quad (1)$$

where the Hessian is given by $H = \partial_{\theta} \partial_{\theta} L|_{\theta=\hat{\theta}}$. The influence on a differentiable function of θ is defined through the chain rule. For example, the influence on the loss of a test data z_{test} is calculated to be

$$\begin{aligned} I^{\text{perf}}(z_{\text{test}}, j) &\stackrel{\text{def}}{=} \left. \frac{dl(z_{\text{test}}, f_{\hat{\theta}_{\epsilon,j}})}{d\epsilon} \right|_{\epsilon=0} \\ &= \left. \frac{dl(z_{\text{test}}, f_{\theta})}{d\theta} \right|_{\theta=\hat{\theta}} \cdot I^{\text{param}}(j). \end{aligned}$$

An underlying assumption is that the Hessian matrix is invertible, which is not always true. Typically, in over-parameterized networks, the loss function often has non-unique minima with flat directions around them. In this paper, we extend the definition of influence functions for cases with a non-invertible Hessian.

Supervised meta-learning. In supervised meta-learning (See Hospedales et al. (2022) for a review), a task $\mathcal{T}$ is defined as the pair of a dataset and a loss function: $(\mathcal{D}^{\mathcal{T}}, \mathcal{L}^{\mathcal{T}})$. The occurrence of each task follows a distribution $\mathcal{T} \sim p(\mathcal{T})$. An adaptation algorithm $\mathcal{A}$ takes as input a task $\mathcal{T}$ and meta-parameters ω , and outputs the weights $\hat{\theta}$ of the model f_{θ} . The learning objective is stated as the optimization of ω with respect to the loss averaged over the task distribution:

$$\hat{\omega} = \underset{\omega}{\operatorname{argmin}} \underset{\mathcal{T} \sim p(\mathcal{T})}{E} [\mathcal{L}^{\mathcal{T}}(\mathcal{D}^{\mathcal{T}}, f_{\hat{\theta}^{\mathcal{T}}})] \quad \text{with } \hat{\theta}^{\mathcal{T}} = \mathcal{A}(\mathcal{T}, \omega)$$

The formulation of empirical risk minimization uses sampled tasks as building blocks, which are divided into a taskset for training (source taskset) $\mathcal{D}^{\text{src}} = \{\mathcal{T}^{\text{src}(i)}\}_{i=1}^M$ and a taskset for testing (target taskset) $\mathcal{D}^{\text{trg}} = \{\mathcal{T}^{\text{trg}(i)}\}_{i=1}^{M'}$. The dataset in each task is also divided into a training and test dataset: $\mathcal{D}^{\mathcal{T}} = (\mathcal{D}^{\mathcal{T}\text{train}}, \mathcal{D}^{\mathcal{T}\text{test}})$. The learning objective is stated as:

$$\hat{\omega} = \underset{\omega}{\operatorname{argmin}} \frac{1}{M} \sum_{i=1}^M \mathcal{L}^{\text{src}(i)}(\mathcal{D}^{\text{src}(i)\text{test}}, f_{\hat{\theta}^i}) \quad \text{with } \hat{\theta}^i = \mathcal{A}(\mathcal{D}^{\text{src}(i)\text{train}}, \mathcal{L}^{\text{src}(i)}, \omega) \quad (2)$$

An example performance metric in meta-testing used in this paper is the test loss $\mathcal{L}^{\text{trg}(i)}(\mathcal{D}^{\text{trg}(i)\text{test}}, f_{\hat{\theta}^i})$ where $\hat{\theta}^i = \mathcal{A}(\mathcal{D}^{\text{trg}(i)\text{train}}, \mathcal{L}^{\text{trg}(i)}, \hat{\omega})$. We experiment with MAML (Finn et al., 2017), and Prototypical Network (Protonet) (Snell et al., 2017), two widely used meta-learning methods. A.1 provides the explicit forms of $\mathcal{A}$ for them.

4 Proposed Method

4.1 Task-level Influence Functions

We now describe our method. TLXML measures the influence of training tasks on the adaptation and inference processes in meta-learning. To measure the influence of a training task $\mathcal{T}^j$ on the model's behaviors, we consider the task-level perturbation of the empirical risk defined in Eq. 2:

$$\hat{\omega}_{\epsilon}^j = \underset{\omega}{\operatorname{argmin}} \frac{1}{M} \sum_{i=1}^M \mathcal{L}^i(\mathcal{D}^{(i)\text{test}}, f_{\hat{\theta}^i}) + \frac{\epsilon}{M} \mathcal{L}^j(\mathcal{D}^{(j)\text{test}}, f_{\hat{\theta}^j}) \quad (3)$$

with $\hat{\theta}^i = \mathcal{A}(\mathcal{D}^{(i)\text{train}}, \mathcal{L}^i, \omega)$. The influence on $\hat{\omega}$ is given by:

$$\begin{aligned} I^{\text{meta}}(j) &\stackrel{\text{def}}{=} \left. \frac{d\hat{\omega}_{\epsilon}^j}{d\epsilon} \right|_{\epsilon=0} \\ &= -\frac{1}{M} H^{-1} \left. \frac{\partial \mathcal{L}^j(\mathcal{D}^{(j)\text{test}}, f_{\hat{\theta}^j})}{\partial \omega} \right|_{\omega=\hat{\omega}} \end{aligned} \quad (4)$$

where the Hessian matrix H is defined as follows:

$$H = \frac{1}{M} \sum_{i=1}^M \left. \frac{\partial^2 \mathcal{L}^i(\mathcal{D}^{(i)\text{test}}, f_{\hat{\theta}^i})}{\partial \omega \partial \omega} \right|_{\omega=\hat{\omega}}. \quad (5)$$

See A.2 for the derivation of Eq. 4. The model's behavior is affected by the perturbation through the adapted parameters $\hat{\theta}_{\epsilon}^{ij} \equiv \mathcal{A}(\mathcal{D}^{(i)\text{train}}, \mathcal{L}^i, \hat{\omega}_{\epsilon}^j)$. The influence of the training task $\mathcal{T}^j$ on model parameters $\hat{\theta}^i = \mathcal{A}(\mathcal{D}^{(i)\text{train}}, \mathcal{L}^i, \hat{\omega})$ is measured by:

$$I^{\text{adpt}}(i, j) \stackrel{\text{def}}{=} \left. \frac{d\hat{\theta}_{\epsilon}^{ij}}{d\epsilon} \right|_{\epsilon=0} \quad (6)$$

$$= \left. \frac{\partial \mathcal{A}(\mathcal{D}^{(i)\text{train}}, \mathcal{L}^i, \omega)}{\partial \omega} \right|_{\omega=\hat{\omega}} I^{\text{meta}}(j) \quad (7)$$

The influence of the training task $\mathcal{T}^j$ on the loss of a test task $\mathcal{T}^i$ is measured by:

$$I^{\text{perf}}(i, j) \stackrel{\text{def}}{=} \left. \frac{d\mathcal{L}^i(\mathcal{D}^{(i)\text{test}}, f_{\hat{\theta}_\epsilon^{ij}})}{d\epsilon} \right|_{\epsilon=0} \quad (8)$$

$$= \left. \frac{\partial \mathcal{L}^i(\mathcal{D}^{(i)\text{test}}, f_\theta)}{\partial \theta} \right|_{\theta=\hat{\theta}^{(i)}} I^{\text{adpt}}(i, j) \quad (9)$$

The training tasks are used only for evaluating I^{meta} . This allows us to obtain other explanation data without needing access to the raw training data, which benefits devices with limited storage capacity. We also note that the above method can be extended to a higher level of abstraction than task-level for situations where task-level explanations are insufficient. See A.3 for details.

4.2 Approximation via Gauss-Newton matrix

TLXML faces computational barriers when applied to large models with its exact Hessian H (Eq. 5). We note that, besides the computational cost of $\mathcal{O}(q^3)$ in inverting H of matrix size q , there is a cost issue in computing H itself, which is peculiar to meta-learning. Although H is defined as the second-order tensor of the meta parameters, the bi-level structure of meta-learning raises a third-order tensor in the form $\partial_\omega \partial_\omega \theta$ during its computation, resulting in a computational cost of $\mathcal{O}(pq^2)$ for a model with p weights and q meta-parameters. (see A.4 for details). To address the above issues, we extend the Gauss-Newton matrix (GN matrix) approximation method established in supervised learning (see, for example, (Martens, 2020)) to a method for meta-learning. For specific loss functions, e.g., mean squared error and cross-entropy, the Hessian can be decomposed into a sum of outer products of two vectors along with terms containing second-order derivatives. In this work, we only work on the case of cross-entropy with the softmax function, which leads to:

$$\frac{\partial^2 L}{\partial \omega \partial \omega} = \text{positive constant} \times \left[\sum_{n,j,k} \sigma_k(\mathbf{y}_n) (\delta_{kj} - \sigma_j(\mathbf{y}_n)) \frac{\partial y_{nk}}{\partial \omega} \frac{\partial y_{nj}}{\partial \omega} - \sum_{n,j,k} t_{nj} (\delta_{jk} - \sigma_k(\mathbf{y}_n)) \frac{\partial^2 y_{nk}}{\partial \omega \partial \omega} \right] \quad (10)$$

where y is the output of the last layer, σ is the softmax function, t is the one-hot vector of the target label, j, k are class indices, and n is the index specifying tasks and input tensors in them. The first term in Eq. 10 is the GN matrix or the Fisher information metric. Since the second-order derivatives in the second term give rise to third-order tensors, we focus on cases where the second-order derivatives are uncorrelated with their coefficients and the Hessian is approximated by the first term. In supervised learning, the loss L and the model output $\mathbf{y}$ are functions of the model's weights θ ; in our setting, they depend on the meta-parameters ω through the adaptation process. Using basic facts regarding cross-entropy (see A.5), we can reuse the argument in supervised learning and show that when the training taskset closely approximates a distribution $P(X|\omega^*)$ with the optimal value ω^* and the learned $\hat{\omega}$ is close to ω^* , the first term in Eq. 10 dominates. As the quadratic form of the gradient vectors for each n in the first term is non-negative, we use its factorized form:

$$\frac{\partial^2 L}{\partial \omega_\mu \partial \omega_\nu} \sim \sum_{n,j} (\mathbf{V})_{\mu(nj)} (\mathbf{V}^T)_{(nj)\nu} \quad (11)$$

where μ and (nj) are regarded as a row index and a column index, respectively. In the case of q meta-parameters, M tasks, N data points per task, and c target classes, the shape of $\mathbf{V}$ is $q \times cNM$, meaning that, if $q > cNM$, the approximated Hessian has zero eigenvalues. This also happens for smaller q if the columns of $\mathbf{V}$ are not independent of each other.

To efficiently compute the inverse (or the pseudo-inverse defined below) of Eq. 11, we orthogonalize the columns of $\mathbf{V}$. In A.6, we show that, under a realistic assumption on the above-mentioned low-rank structure, this can be done sequentially by repeatedly orthogonalizing small sets of column vectors, avoiding the memory and computational costs of accumulating all terms and orthogonalizing a large set of vectors at once.

4.3 Influence Functions with Flat Directions

The appearance of a low-rank structure of the Hessian is not limited to the cases of the approximation mentioned above. Generally, it appears when the constraints imposed by the training tasks on the meta-parameters are not enough

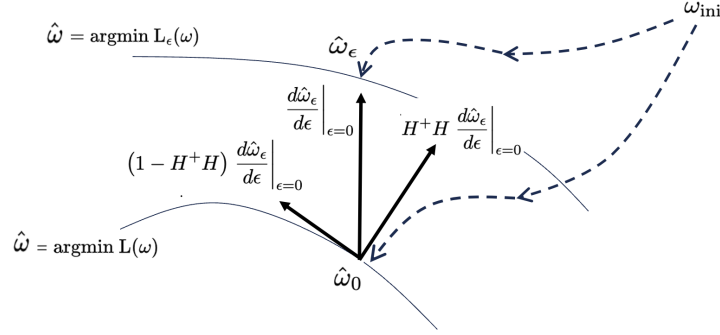


Figure 2: Diagram of the projected influence function, which measures the influence of a training task on the meta-parameters with the Hessian flat directions projected out.

to determine the unique optimal values. Figure 2 depicts a generic case where flat directions of the Hessian appear in the parameter space. When the number of parameters is sufficiently large, the points that satisfy the minimization condition, $\hat{\omega} = \text{argmin } L(\omega)$, form a hypersurface, resulting in flat directions of the Hessian. The same holds for the perturbed loss $L_\epsilon(\omega)$ used for defining the influence functions. The position along the flat directions resulting from minimization depends on the initial conditions and the learning algorithm. We do not explore this dependency in our paper. Instead, we employ a geometric definition of influence functions: we take the partial inverse H^+ of H in the subspace perpendicular to the flat directions, known as the pseudo-inverse matrix. With this approach, we modify the definitions of influence functions as:

$$\begin{aligned} I^{\text{meta}}(j) &\stackrel{\text{def}}{=} H^+H \frac{d\hat{\omega}_\epsilon^j}{d\epsilon} \Big|_{\epsilon=0} \\ &= -\frac{1}{M} H^+ \frac{\partial \mathcal{L}^j(D^{(j)\text{test}}, f_{\hat{\theta}_j})}{\partial \omega} \Big|_{\omega=\hat{\omega}}. \end{aligned} \quad (12)$$

See A.2 for the derivation of the second equation. H^+H represents the projection that drops the flat directions. When the Fisher information metric approximates the Hessian, the data distribution is unchanged in the flat directions, and the influence function projected by H^+H reflects the sensitivity of $\hat{\omega}$ in the steepest direction of distribution change. We also note that with that approximation (Eq. 11), H^+ can also be written as a summation of factorized forms. This property is crucial for implementation. See A.6 for details.

In supervised learning, it is argued that the non-convexity of loss functions causes erroneous influence functions (Basu et al., 2021). The remedy of influence functions with the pseudo-inverse Hessian will also be effective in addressing that common issue.

4.4 One-step Update via TLXML

We consider utilizing TLXML to enhance the performance of meta-learners. In supervised learning, Koh and Liang (2017) employed a leave-out approach, in which the network is retrained after removing training data with low influence scores. Applying this approach to improve meta-learning seems reasonable, but it digresses from our theoretical argument. Since the influence functions are defined based solely on the local structure around the convergence point, it is not guaranteed that tasks or data with low scores exert negative influences throughout the entire training process.

Recalling that I^{meta} represents the derivative of the meta-parameters with respect to the perturbation ϵ in the training task distribution, we can regard it as a linear approximation of the parameter shifts caused by that distribution change:

$$\delta\omega \sim \xi \times \frac{d\hat{\omega}_\epsilon^j}{d\epsilon} \Big|_{\epsilon=0} = \xi \times I^{\text{meta}}(j) \quad (13)$$

where we renamed the non-zero perturbation parameter to ξ to avoid confusion with the differential variable. The case of $\xi = -1$ corresponds to removing task j , and if we are agnostic about the difference between supervised-learning and meta-learning, it is the situation considered by Koh and Liang (2017); Guo et al. (2020); Golatkar et al. (2020, 2021) for removing the influence of specific training data points. However, we should also note that ξ can take other values as a parameter of the training task distribution. Eq. 13 can be used not only to remove the influence of low-scored tasks but also to adjust it with other negative values of ξ , and even amplify the influence of high-scored tasks with positive ξ without rerunning the training.

5 Experiments

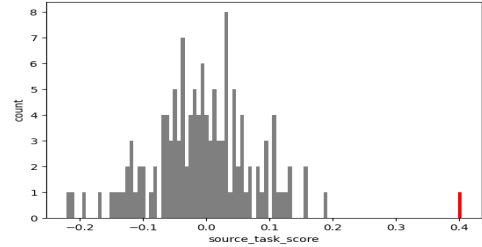
We first examine whether TLXML provides adequate explanations that attribute the model’s behavior to the influence of meta-learning training tasks, and second, we examine whether the post-meta-training update using them improves the adaptation ability.

5.1 Validation of TLXML

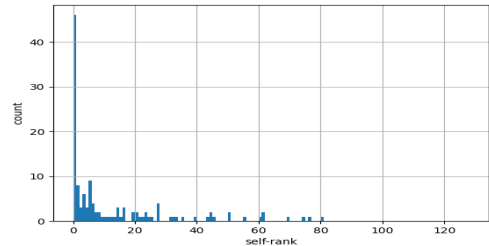
We empirically investigate whether the proposed method satisfies two fundamental properties: **1)** If the network memorizes a training task, its influence on a test task with similar characteristics should be scored higher than the influence of other training tasks; and **2)** if the network encodes generalizable information about the task distribution, training tasks that belong to a subpopulation sharing salient features with the test task, should receive higher scores than tasks outside that subpopulation. Both properties are natural requirements for an explanation method to be regarded as a method based on past experiences, as training tasks that resemble the test task generally boost test performance. We employ MAML and Protonet as meta-learning algorithms and conduct experiments with few-shot learning problems taken from the MiniImagenet (Vinyals et al., 2016), Ominiglot (Lake et al., 2015) datasets, and two newly defined datasets. Unless otherwise stated, experiments use a 5-way 5-shot configuration. The implementation builds on the *lean2learn* meta-learning library (Arnold et al., 2020) and PyTorch’s functions for automatic differentiation. We train the meta-parameters with Adam, employing a meta-batch size of 32.

5.1.1 Distinction of Tasks

To validate property 1, we generate pairs of similar training and test tasks by making each test task identical to one of the training tasks. We then apply Eq. 9 to the trained network and examine whether it assigns the highest influence score to the training task identical to each test task. Figure 3 presents the results for a two-layer, fully connected network (1,285 parameters) overfitted to 128 MiniImageNet training tasks with MAML. Figure 3a illustrates a successful case in which the training task identical to the test task (highlighted in red) is clearly distinguished from all other training tasks. Figure 3b shows the distribution of the ranks assigned to the training tasks with this kind across 128 tests (we call them *self-ranks*). Although they often appear near the top of the ranking², it is not always placed first (it is 12.6 ± 18.9). This is likely because of the non-convexity of the training loss. In our case, many of the 1285 Hessian eigenvalues are close to zero, and 92 are negative, violating the underlying assumption of Eq. 4. The extended influence function in Eq. 12, which replaces the inverse Hessian with its pseudo-inverse, circumvents this instability. Replacing the 92 negative eigenvalues with zero and inverting the only 1193 positive eigenvalues drives the self-rank to a perfect 0.0 ± 0.0 . The rank stays perfect as we aggressively truncate the spectrum down to 1024, 512, 256, 128, and 64. When we truncate it to 32, 16, or 8 eigenvalues, the self-ranks degrade (0.0 ± 0.2 , 2.0 ± 3.2 , and 8.6 ± 9.1 , respectively). This underscores the need for an appropriate estimate of the Hessian’s eigenspace to maintain reliable task discrimination. See B.2 for details.



(a) Example of training task score distribution in a single test. Highlighted is the training task identical to the test task.



(b) Histogram of the self-ranks. The self-rank is defined as the rank of the training task identical to each test task.

Figure 3: Test with training tasks with a two-layer fully connected network (1,285 parameters) overfitted to 128 training tasks in MiniImageNet with MAML.

²In B.2, as a sanity check, we see that reducing the similarity of training and test tasks leads to a degradation of the self-ranks.

Table 1: Experiment of distinguishing noise tasks and normal task distributions. MI and OM represent MiniImagenet and Omniglot respectively. The standard deviation σ under the hypothesis of the random ordering is $\sqrt{0.5 \times 0.5 \times 128} \sim 5.66$. $\pm$ means the average and standard deviation across 5 runs of training.

learning method	dataset	proper tests	
		[count]	$[\sigma]$
MAML	MI	85.6 $\pm$ 11.1	3.8 $\pm$ 2.0 σ
MAML	OM	94.0 $\pm$ 6.0	5.3 $\pm$ 1.1 σ
Protonet	MI	95.6 $\pm$ 5.3	5.6 $\pm$ 0.9 σ
Protonet	OM	81.2 $\pm$ 5.5	3.0 $\pm$ 1.0 σ

5.1.2 Distinction of Normal and Noise Task Distributions

To validate property 2, we prepare three training tasksets. Each set includes a subpopulation with less similarity to the test tasks than the rest of the training tasks, which is achieved by replacing a subset of a base taskset with tasks consisting of noise data.

Classification of Synthetic Data To examine the method with the exact form of Eq. 4, we employ a lightweight network and created tasksets of clustered data points in a two-dimensional space that the network can learn easily. Using Gaussian distributions, we first sample cluster centers around the origin of the plane and then the members of each cluster around its center. Noise tasks consist of data points sampled around the origin and assigned random labels. Figure 4 shows the results of a 3-layer fully connected network (63 parameters) trained with 1024 training tasks including 128 noise tasks in the 3-ways-5-shots problem setting. We observe that the normal and noise tasks follow different distributions, and the former is scored higher, as was expected intuitively. We refer to the relation between two distributions that is aligned with the intuitive order of similarity to the test task as *proper order*, and the tests that result in that relation as *tests with the proper order* or *proper tests*. We score the training tasks with the influence function for 128 test losses, and observe that 113 tests resulted in the proper order in terms of the mean values of the subpopulations. According to a two-sided binomial test with the null hypothesis of random ordering, this count exceeds the average count ($= 64$) of the binomial distribution by 8.7σ , satisfying property 2 statistically.

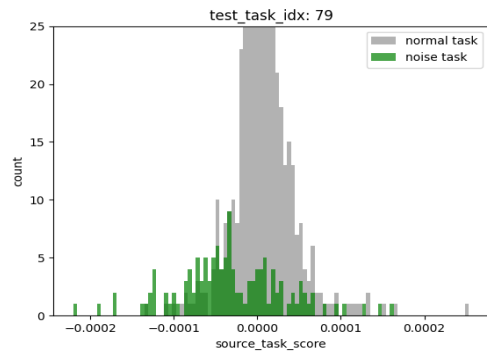


Figure 4: Example of training task score distribution (synthetic dataset).

MiniImagenet and Omniglot. We validate property 2 on realistic tasks by using a large network, for which we adopt the approximation method in Eq. 11. For each dataset, using 8192 training tasks and replacing the image tensors of 1024 tasks with uniform noise tensors of the same shape (noise images). We train a network with 3-conv layers using MAML ($\sim 20k$ parameters) and a 4-conv-layer feature extractor using Protonet ($\sim 28k$ parameters). For each training setup, we evaluate the influence of the training tasks on 128 test loss values using Eq. 9 with the projected influence on the meta-parameters (Eq. 12) and the GN matrix approximation (Eq. 11). Table 1 shows the numbers of proper tests; Again, the binomial test suggests that our scoring method satisfies property 2. See B.3 for the results with different training settings.

5.1.3 Consistency with Semantic Similarity

We also validate property 2 in terms of semantic similarities between training and test tasks. We define a new image dataset, *FC60*, in which each image is assigned hierarchical labels specifying the super- and sub-classes. The train/test splits 60 subclasses in total, and semantic similarities between train and test tasks are built in by 12 superclasses shared across the splits. This structure is realized by splitting the subclasses of each superclass in the training taskset of the FC100 dataset (Oreshkin et al., 2018). Figure 5 shows examples of a test and training task in the FC60 dataset. We train a 3-conv-layer network using MAML and a 4-conv-layer feature extractor using Protonet with 8192 training tasks from FC60. Table 2 shows the number of proper tests with the training subpopulation defined by how many superclasses it

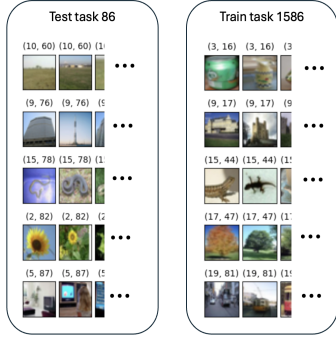


Figure 5: Examples of FC60 tasks. The pair of labels on each image represents the semantic labels (superclass, subclass) obtained from CIFAR100.

Figure 6: Experiment of distinguishing training tasks with superclasses shared with test tasks from other training tasks. $\pm$ means the average and standard deviation across 1024 test tasks. The standard deviation σ under the hypothesis of the random ordering is $\sqrt{0.5 \times 0.5 \times 1024} = 16$.

learning method	overlap	train tasks filtered	proper tests
MAML	1	7522 $\pm$ 314	618 (+6.6 σ)
MAML	2	4781 $\pm$ 718	676 (+10.3 σ)
MAML	3	1508 $\pm$ 422	688 (+11.0 σ)
MAML	4	171 $\pm$ 69	637 (+7.8 σ)
Protonet	1	7522 $\pm$ 314	658 (+9.1 σ)
Protonet	2	4781 $\pm$ 718	672 (+10.0 σ)
Protonet	3	1508 $\pm$ 422	703 (+11.9 σ)
Protonet	4	171 $\pm$ 69	644 (+8.3 σ)

Table 2: Experiment of distinguishing training tasks with superclasses shared with test tasks from other training tasks. $\pm$ means the average and standard deviation across 1024 test tasks. The standard deviation σ under the hypothesis of the random ordering is $\sqrt{0.5 \times 0.5 \times 1024} = 16$.

learning method	overlap	train tasks filtered	proper tests
MAML	1	7522 $\pm$ 314	618 (+6.6 σ)
MAML	2	4781 $\pm$ 718	676 (+10.3 σ)
MAML	3	1508 $\pm$ 422	688 (+11.0 σ)
MAML	4	171 $\pm$ 69	637 (+7.8 σ)
Protonet	1	7522 $\pm$ 314	658 (+9.1 σ)
Protonet	2	4781 $\pm$ 718	672 (+10.0 σ)
Protonet	3	1508 $\pm$ 422	703 (+11.9 σ)
Protonet	4	171 $\pm$ 69	644 (+8.3 σ)

shares with the test task, illustrating that TLXML distinguishes training tasks that share semantic properties with test tasks from the other tasks. See B.4 for details.

5.2 Post-Convergence One-Step Update

We examine the effect of the one-step update Eq. 13 from the convergence point of meta-learning. We utilize 8192 training tasks from FC60 and train a network with 3-conv layers using MAML. Each training task is scored with the influence function for the average test loss across 1024 test tasks. Table 3 shows the effects of blocking the 2048 lowest-score training tasks with one-step updates (and leave-out for comparison) on the test accuracies. We observe that the test accuracies are improved by one-step update with suitable shift values ξ , whereas leave-out retraining fails to yield a statistically significant gain. Table 4 shows the effects of both blocking the 2024 lowest-score and enhancing the 2024 highest-score training tasks in two cases: test datasets with and without shared superclasses (FC60 testset and FC100 testset, respectively). We observe improvement in three of the four test settings. Although the case of enhancing the training tasks for the FC100 testset is an exception, we can see that other one-step update parameters boost performance in that case(see B.5 for details).

6 Discussion and Conclusion

This paper introduced TLXML, a method for quantifying the impact of meta-learning tasks. We reduced its computational cost using the GN matrix approximation and handled flat directions around the convergence point using the pseudo-inverse Hessian. Experiments suggest that TLXML offers explanations at the task level properly, leading to actionable insights and improved performance in downstream tasks.

Extending TLXML beyond classification tasks, e.g., regression and Reinforcement Learning (RL), is expected to be straightforward, as its formulation through gradient is almost agnostic to the differences in the learning objectives. For example, it is plausible that an RL policy can be improved using a similar technique. Moreover, because TLXML possesses an aspect of task similarity, future work should explore its relation to general notions, such as task embedding

Table 3: Test accuracies after a single TLXML-guided update to a MAML model with blocked tasks. $\pm$ means the average and standard deviation across 5 runs of MAML training. Bold values outperform the MAML baseline (shown in the bracket) with a unpaired Welch two-sample t -test ($p < 0.05$).

test taskset	# tasks	Method	Accuracy
FC60	2048	$\xi = -8$	0.659 ± 0.009
(0.663 $\pm$ 0.004)	2048	$\xi = -4$	0.674 $\pm$ 0.007
	2048	$\xi = -2$	0.672 $\pm$ 0.007
	2048	$\xi = -1$	0.669 ± 0.004
	2048	leave-out	0.667 ± 0.01

Table 4: Test accuracies after a single TLXML-guided update to a trained model with blocked and enhanced tasks. See Table 3 for notation.

test taskset	# tasks	$\xi = -4$ (block)
FC60 (0.663 $\pm$ 0.004)	2048	0.674 $\pm$ 0.007
FC100 (0.429 $\pm$ 0.005)	2048	0.449 $\pm$ 0.011
test taskset	# tasks	$\xi = 4$ (enhance)
FC60 (0.663 $\pm$ 0.004)	2048	0.676 $\pm$ 0.008
FC100 (0.429 $\pm$ 0.05)	2048	0.429 ± 0.010

space, out-of-distribution awareness, or domain adaptations, not just the usage as a similarity measure between training and test tasks in each single dataset.

One limitation is the assumption of a local minimum of the loss function in the definitions of the influence functions. Future work could aim to design influence functions compatible with early stopping techniques. We also note that the current evaluation is a purely quantitative one conducted with standard benchmark datasets; investigating how non-experts would leverage TLXML is also a worthwhile future direction.

Acknowledgments

The authors are grateful to Sheila Schoepp for helpful discussions at the early stages of this work and warm assistance with improving and formatting draft versions.

References

- Adebayo, J., Muelly, M., Abelson, H., and Kim, B. (2022). Post hoc explanations may be ineffective for detecting unknown spurious correlation. In *International Conference on Learning Representations (ICLR)*.
- Agarwal, M., Yurochkin, M., and Sun, Y. (2021). On sensitivity of meta-learning to support data. In *Conference on Neural Information Processing Systems (NeurIPS)*.
- Alaa, A. M. and van der Schaar, M. (2020). Discriminative jackknife: Quantifying uncertainty in deep learning via higher-order influence functions. In *International Conference on Machine Learning (ICML)*.
- Arnold, S. M. R., Mahajan, P., Datta, D., Bunner, I., and Zarkias, K. S. (2020). learn2learn: A library for meta-learning research. *arXiv preprint arXiv:2008.12284*.
- Barshan, E., Brunet, M., and Dziugaite, G. K. (2020). Relatif: Identifying explanatory training samples via relative influence. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*.
- Basu, S., Pope, P., and Feizi, S. (2021). Influence functions in deep learning are fragile. In *9th International Conference on Learning Representations (ICLR), Virtual Event, Austria, May 3-7, 2021*. OpenReview.net.
- Cao, Y. and Yang, J. (2015). Towards making systems forget with machine unlearning. In *IEEE IEEE Symposium on Security and Privacy*.
- Chhabra, A., Li, P., Mohapatra, P., and Liu, H. (2024). "what data benefits my classifier?" enhancing model performance and interpretability through influence-based data selection. In *International Conference on Learning Representations (ICLR)*.
- Cohen, G., Sapiro, G., and Giryas, R. (2020). Detecting adversarial samples using influence functions and nearest neighbors. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

- Cook, R. D. and Weisberg, S. (1980). Characterizations of an empirical influence function for detecting influential cases in regression. *Technometrics*, 22(4):495–508.
- Csurka, G., Dance, C. R., Fan, L., Willamowski, J., and Bray, C. (2002). Visual categorization with bags of keypoints. In *European Conference on Computer Vision (ECCV)*.
- Finn, C., Abbeel, P., and Levine, S. (2017). Model-agnostic meta-learning for fast adaptation of deep networks. In *International Conference on Machine Learning (ICML)*.
- Golatkhar, A., Achille, A., Ravichandran, A., Polito, M., and Soatto, S. (2021). Mixed-privacy forgetting in deep networks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Golatkhar, A., Achille, A., and Soatto, S. (2020). Eternal sunshine of the spotless net: Selective forgetting in deep networks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Goldblum, M., Fowl, L., and Goldstein, T. (2020). Adversarially robust few-shot learning: A meta-learning approach. In *Conference on Neural Information Processing Systems (NeurIPS)*.
- Grosse, R., Bae, J., Anil, C., Elhage, N., Tamkin, A., Tajdini, A., Steiner, B., Li, D., Durmus, E., Perez, E., et al. (2023). Studying large language model generalization with influence functions. *arXiv preprint arXiv:2308.03296*.
- Guo, C., Goldstein, T., Hannun, A., and van der Maaten, L. (2020). Certified data removal from machine learning models. In *International Conference on Machine Learning (ICML)*.
- Hampel, F. R. (1974). The influence curve and its role in robust estimation. *Journal of the American Statistical Association*, 69(346):383–393.
- Han, X. and Tsvetkov, Y. (2020). Fortifying toxic speech detectors against veiled toxicity. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Han, X., Wallace, B. C., and Tsvetkov, Y. (2020). Explaining black box predictions and unveiling data artifacts through influence functions. In *Annual Meeting of the Association for Computational Linguistics (ACL)*.
- Hospedales, T., Antoniou, A., Micaelli, P., and Storkey, A. (2022). Meta-learning in neural networks: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(9):5149–5169.
- Khanna, R., Kim, B., Ghosh, J., and Koyejo, O. (2019). Interpreting black box predictions using fisher kernels. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*.
- Khattar, V., Ding, Y., Sel, B., Lavaei, J., and Jin, M. (2023). A cmdp-within-online framework for meta-safe reinforcement learning. In *International Conference on Learning Representations (ICLR)*.
- Koh, P. W., Ang, K., Teo, H. H. K., and Liang, P. (2019). On the accuracy of influence functions for measuring group effects. In *Conference on Neural Information Processing Systems (NeurIPS)*.
- Koh, P. W. and Liang, P. (2017). Understanding black-box predictions via influence functions. In *International Conference on Machine Learning (ICML)*.
- Krizhevsky, A. (2009). Learning multiple layers of features from tiny images.
- Lake, B. M., Salakhutdinov, R., and Tenenbaum, J. B. (2015). Human-level concept learning through probabilistic program induction. *Science*, 350(6266):1332–1338.
- Li, D., Yang, Y., Song, Y., and Hospedales, T. M. (2018). Learning to generalize: Meta-learning for domain generalization. In *AAAI Conference on Artificial Intelligence (AAAI)*.
- Li, N., Zhou, C., Gao, Y., Chen, H., Zhang, Z., Kuang, B., and Fu, A. (2025). Machine unlearning: Taxonomy, metrics, applications, challenges, and prospects. *IEEE Transactions on Neural Networks and Learning Systems*, 36(8):13709–13729.
- Lin, Z., Thomas, G., Yang, G., and Ma, T. (2020). Model-based adversarial meta-reinforcement learning. In *Conference on Neural Information Processing Systems (NeurIPS)*.
- Lowe, D. G. (1999). Object recognition from local scale-invariant features. In *International Conference on Computer Vision (ICCV)*.
- Lowe, D. G. (2004). Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110.
- Lu, C., Wu, Y., Hernández-Lobato, J. M., and Schölkopf, B. (2021). Invariant causal representation learning for out-of-distribution generalization. In *International Conference on Learning Representations (ICLR)*.
- Mann, K. S., Schneider, S., Chiappa, A., Lee, J. H., Bethge, M., Mathis, A., and Mathis, M. W. (2021). Out-of-distribution generalization of internal models is correlated with reward. In *Self-Supervision for Reinforcement Learning Workshop, International Conference on Learning Representations (ICLR)*.

- Martens, J. (2020). New insights and perspectives on the natural gradient method. *Journal of Machine Learning Research*, 21(146):1–76.
- Martens, J. and Grosse, R. B. (2015). Optimizing neural networks with kronecker-factored approximate curvature. In Bach, F. R. and Blei, D. M., editors, *32nd International Conference on Machine Learning(ICML)*. JMLR.org.
- Mouli, S. C., Alam, M. A., and Ribeiro, B. (2024). Metaphysica: Improving OOD robustness in physics-informed machine learning. In *International Conference on Learning Representations (ICLR)*.
- Ni, R., Goldblum, M., Sharaf, A., Kong, K., and Goldstein, T. (2021). Data augmentation for meta-learning. In *International Conference on Machine Learning (ICML)*.
- Oreshkin, B. N., Rodriguez, P., and Lacoste, A. (2018). Tadam: Task dependent adaptive metric for improved few-shot learning. In *Conference on Neural Information Processing Systems (NeurIPS)*.
- Ribeiro, M. T., Singh, S., and Guestrin, C. (2016). "why should i trust you?": Explaining the predictions of any classifier. In *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*.
- Shao, X., Wang, H., Zhu, X., Xiong, F., Mu, T., and Zhang, Y. (2023). Effect: Explainable framework for meta-learning in automatic classification algorithm selection. *Information Sciences*, 622:211–234.
- Shu, Y., Cao, Z., Wang, C., Wang, J., and Long, M. (2021). Open domain generalization with domain-augmented meta-learning. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Sijben, E., Jansen, J. C., Bosman, P. A. N., and Alderliesten, T. (2024). Function class learning with genetic programming: Towards explainable meta learning for tumor growth functionals. In *Genetic and Evolutionary Computation Conference (GECCO)*.
- Snell, J., Swersky, K., and Zemel, R. (2017). Prototypical networks for few-shot learning. In *Conference on Neural Information Processing Systems (NeurIPS)*.
- Song, Y. and Jeong, H. (2024). Towards cross-domain generalization of hamiltonian representation via meta-learning. In *International Conference on Learning Representations (ICLR)*.
- Vinyals, O., Blundell, C., Lillicrap, T., Kavukcuoglu, K., and Wierstra, D. (2016). Matching networks for one shot learning. In *Conference on Neural Information Processing Systems (NeurIPS)*.
- Wen, L., Zhang, S., Tseng, H. E., Singh, B., Filev, D., and Peng, H. (2022). Improved robustness and safety for pre-adaptation of meta-reinforcement learning with prior regularization. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*.
- Woźnica, K. and Biecek, P. (2021). Towards explainable meta-learning. In *European Conference on Machine Learning and Principles and Practice of Knowledge Discovery in Databases (ECML PKDD)*.
- Xu, H., Li, Y., Liu, X., Liu, H., and Tang, J. (2021). Yet meta learning can adapt fast, it can also break easily. In *SIAM International Conference on Data Mining (SDM)*.
- Yao, Y., Liu, Z., Cen, Z., Zhu, J., Yu, W., Zhang, T., and Zhao, D. (2023). Constraint-conditioned policy optimization for versatile safe reinforcement learning. In *Conference on Neural Information Processing Systems (NeurIPS)*.
- Zhang, J., Cheung, B., Finn, C., Levine, S., and Jayaraman, D. (2020). Cautious adaptation for reinforcement learning in safety-critical settings. In *International Conference on Machine Learning (ICML)*.
- Zügner, D. and Günnemann, S. (2019). Adversarial attacks on graph neural networks via meta learning. In *International Conference on Learning Representations (ICLR)*.

A Method details

A.1 Meta-learning Algorithms

We present the explicit forms of the adaptation algorithm $\mathcal{A}$ used for MAML and Protonet.

In MAML, the initial values θ_0 of the network weights serve as the meta-parameters, and $\mathcal{A}$ represents a one-step gradient descent update of the weights with a fixed learning rate α :

$$\mathcal{A}(\mathcal{D}^{(i)\text{train}}, \mathcal{L}^i, \theta_0) = \theta_0 - \alpha \left. \partial_{\theta} \mathcal{L}^i(\mathcal{D}^{(i)\text{train}}, f_{\theta}) \right|_{\theta=\theta_0}.$$

In Protonet, the meta-parameters are the weights of a feature extractor f_{θ} and the adaptation $\mathcal{A}$ does not involve the loss function. It passes the weights θ without any modification and calculates the feature centroid c_k for each class k based on the support set $S_k \in \mathcal{D}$:

$$\begin{aligned} \theta &= \mathcal{A}_{\theta}(\mathcal{D}^{(i)\text{train}}, \mathcal{L}^i, \theta), \\ c_k &= \mathcal{A}_k(\mathcal{D}^{(i)\text{train}}, \mathcal{L}^i, \theta) = \frac{1}{|S_k^{(i)}|} \sum_{(x,y) \in S_k^{(i)}} f_{\theta}(x). \end{aligned}$$

The class prediction for each data point x is given by the following weighted distances to the centroids:

$$P_{\theta}(l|x) = \frac{\exp d(f_{\theta}(x), c_l)}{\sum_k \exp d(f_{\theta}(x), c_k)},$$

where d is a distance measure(e.g. Euclidean distance).

A.2 Implicit differentiation

The second equation in each of Eq. 1, Eq. 4, and Eq. 12 are derived immediately from the following property.

Proposition 1. If a vector parameter $\hat{\theta}_{\epsilon}$ is parametrized by a scalar parameter ϵ in a way that the local maximum or the local minimum condition of a second-order differentiable function $L(\theta, \epsilon)$ is satisfied for each value of ϵ , then the derivative of $\hat{\theta}_{\epsilon}$ with respect to ϵ satisfies:

$$\left. \frac{\partial^2 L(\theta, \epsilon)}{\partial \theta \partial \theta} \right|_{\theta=\hat{\theta}_{\epsilon}} \frac{d\hat{\theta}_{\epsilon}}{d\epsilon} = - \left. \frac{\partial L(\theta, \epsilon)}{\partial \theta \partial \epsilon} \right|_{\theta=\hat{\theta}_{\epsilon}}. \quad (14)$$

Proof. The proof is done almost trivially by differentiating the local maximum or minimum condition

$$\left. \frac{\partial L(\theta, \epsilon)}{\partial \theta} \right|_{\theta=\hat{\theta}_{\epsilon}} = 0 \quad (15)$$

with respect to ϵ and apply the chain rule. $\square$

Note that if the matrix $\partial \partial L$ in Eq. 14 is invertible, we can solve the equation to obtain the ϵ -derivative of $\hat{\theta}_{\epsilon}$. Note also that we do not assume $\hat{\theta}_{\epsilon}$ to be the unique solution of Eq. 15 and Eq. 14 is true for any parametrization of θ with ϵ that satisfies Eq. 15.

A.3 Task Grouping

In some cases, the abstraction of task-level explanations is insufficient, and explanations based on task groups are more suitable. This requirement occurs when some tasks in the training taskset are too similar to each other for human intuition. For example, when an image recognition model is trained with task augmentation(see Ni et al. (2021) for terminology of data augmentation for meta-learning), e.g., flipping, rotating, or distorting the images in original tasks, the influence of each deformed task is not of interest; rather, the influence of the task group generated from each original task is of interest.

We extend the definition of influence functions to the task-group level by considering a common perturbation ϵ in the losses of tasks within a task group $\mathcal{G}^J = \{\mathcal{T}^{j_0}, \mathcal{T}^{j_1}, \dots\}$:

$$\hat{\omega}_\epsilon^J = \arg \min_{\omega} \frac{1}{M} \left[\sum_{i=1}^M \mathcal{L}^i(\mathcal{D}^{(i)\text{test}}, f_{\hat{\theta}^i}) + \epsilon \sum_{\mathcal{T}^j \in \mathcal{G}^J} \mathcal{L}^j(\mathcal{D}^{(j)\text{test}}, f_{\hat{\theta}^j}) \right] \quad (16)$$

with $\hat{\theta}^i = \mathcal{A}(\mathcal{D}^{(i)\text{train}}, \mathcal{L}^i, \omega)$,

which modifies the influence function in Eq. 4 as:

$$I^{\text{meta}}(J) \stackrel{\text{def}}{=} \left. \frac{d\hat{\omega}_\epsilon^J}{d\epsilon} \right|_{\epsilon=0} = \sum_{\mathcal{T}^j \in \mathcal{G}^J} I^{\text{meta}}(j). \quad (17)$$

Eq. 7 and Eq. 9 are only affected by replacing the task index j with the task-group index J . The derivation of Eq. 17 is done by directly applying the argument in A.2

A.4 Third-order tensors in influence functions

Here, we explain how the computational cost of $\mathcal{O}(pq^2)$ arises in evaluating the influence function in Eq. 4. This is due to the third-order tensors which appear in the intermediate process of evaluating the Hessian:

$$H = \frac{1}{M} \sum_{i=1}^M \frac{\partial^2 \mathcal{L}^i(\mathcal{D}^{(i)\text{test}}, f_{\hat{\theta}^i(\omega)})}{\partial \omega \partial \omega} \quad (18)$$

$$\begin{aligned} &= \frac{1}{M} \sum_{i=1}^M \left[\left(\frac{\partial \hat{\theta}^i(\omega)}{\partial \omega} \right)^T \frac{\partial^2 \mathcal{L}^i(\mathcal{D}^{(i)\text{test}}, f_{\theta})}{\partial \theta \partial \theta} \Big|_{\theta=\hat{\theta}^i(\omega)} \frac{\partial \hat{\theta}^i(\omega)}{\partial \omega} \right. \\ &\quad \left. + \frac{\partial \mathcal{L}^i(\mathcal{D}^{(i)\text{test}}, f_{\theta})}{\partial \theta} \Big|_{\theta=\hat{\theta}^i(\omega)} \frac{\partial^2 \hat{\theta}^i(\omega)}{\partial \omega \partial \omega} \right] \quad (19) \end{aligned}$$

where $\hat{\theta}^i = \mathcal{A}(\mathcal{L}^{(i)\text{train}}, \mathcal{L}^i, \omega)$. Because $\hat{\theta}^i$ and ω are p -dimensional and q -dimensional respectively, the second-order derivative $\partial \partial \hat{\theta}^i$ in the last term is the third-order tensor of pq^2 elements. This tensor also appears in the evaluation of the second-order derivative of the network output with respect to ω . In the case of MAML, this tensor is in the form of a third-order derivative:

$$\begin{aligned} \frac{\partial^2 \hat{\theta}^i(\theta_0)}{\partial \theta_0 \partial \theta_0} &= \frac{\partial^2}{\partial \theta_0 \partial \theta_0} \mathcal{A}(\mathcal{D}^{(i)\text{train}}, \mathcal{L}^i, \theta_0) \\ &= -\alpha \frac{\partial^3}{\partial \theta_0 \partial \theta_0 \partial \theta_0} \mathcal{L}^i(\mathcal{D}^{(i)\text{train}}, f_{\theta_0}). \end{aligned}$$

A.5 Relations Among KL-divergence, Cross-Entropy, and Fisher Information Metric

For the reader's convenience, we present basic facts related to the approximation method argued in Section 4.2.

Variant expressions of Hessian. The cross-entropy L between two probability distributions $P(X|\omega^*)$, $P(X|\omega)$ parametrized by ω^* and ω , is equivalent to the Kullback–Leibler (KL) divergence up to a ω -independent term:

$$D_{KL}(P(X|\omega^*), P(X|\omega)) = L(P(X|\omega^*), P(X|\omega)) + \int P(X|\omega^*) \log P(X|\omega^*) dX.$$

Therefore, the second-order derivatives of them with respect to ω are identical:

$$\frac{\partial^2}{\partial \omega \partial \omega} D_{KL} = \frac{\partial^2}{\partial \omega \partial \omega} L$$

Furthermore, considering the Taylor expansion of D_{KL} with $\Delta\omega = \omega - \omega^*$

$$\begin{aligned}
D_{KL}(P(X|\omega^*), P(X|\omega)) &\sim - \sum_{\mu} \Delta\omega^{\mu} \left[\int \partial_{\mu} P(X|\omega^*) dX \right] \\
&\quad + \frac{1}{2} \sum_{\mu\nu} \Delta\omega^{\mu} \Delta\omega^{\nu} \left[\int -\partial_{\mu} \partial_{\nu} P(X|\omega^*) + \frac{\partial_{\mu} P(X|\omega^*) \partial_{\nu} P(X|\omega^*)^2}{P(X|\omega^*)} dX \right] \\
&= \frac{1}{2} \sum_{\mu\nu} \Delta\omega^{\mu} \Delta\omega^{\nu} \mathbb{E}_{X \sim P(X|\omega^*)} [\partial_{\mu} \log P(X|\omega^*) \partial_{\nu} \log P(X|\omega^*)] \\
&\equiv \frac{1}{2} \sum_{\mu\nu} g_{\mu\nu}(\omega^*) \Delta\omega^{\mu} \Delta\omega^{\nu},
\end{aligned}$$

where $g_{\mu\nu}$ is the Fisher information metric, we see that

$$\frac{\partial^2}{\partial\omega_{\mu}\partial\omega_{\nu}} D_{KL} \Big|_{\omega=\omega^*} = \frac{\partial^2}{\partial\omega_{\mu}\partial\omega_{\nu}} L \Big|_{\omega=\omega^*} = g_{\mu\nu}(\omega^*) \quad (20)$$

Approximations by empirical sums. Let us consider the case that $X = (x, c)$ is the pair of a network input x_n and a class label c_n and P is the composition of soft-max function σ and the network output $\mathbf{y}_n \equiv f_{\omega}(x_n)$. Assuming that sampled data accurately approximate the distributions, we obtain the expressions of the Fisher information metric in the form of an empirical sum:

$$\begin{aligned}
g_{\mu\nu}(\omega^*) &= \mathbb{E}_{X \sim P(X|\omega^*)} [\partial_{\mu} \log P(X|\omega^*) \partial_{\nu} \log P(X|\omega^*)] \\
&= \mathbb{E}_{(c,x) \sim P_{\omega^*}(c|x)P(x)} [\partial_{\mu} \log (P_{\omega}(c|x)P(x)) \partial_{\nu} \log (P_{\omega}(c|x)P(x))] \Big|_{\omega=\omega^*} \\
&= \mathbb{E}_{(c,x) \sim P_{\omega^*}(c|x)P(x)} [\partial_{\mu} \log (P_{\omega}(c|x)) \partial_{\nu} \log (P_{\omega}(c|x))] \Big|_{\omega=\omega^*} \\
&\sim \sum_{ni} \sigma_i(\mathbf{y}_n) [\partial_{\mu} \log (\sigma_i(\mathbf{y}_n)) \partial_{\nu} \log (\sigma_i(\mathbf{y}_n))] \Big|_{\omega=\omega^*} \\
&= \sum_{nijk} \sigma_i(\mathbf{y}_n) (\delta_{ik} - \sigma_k(\mathbf{y}_n)) (\delta_{ij} - \sigma_j(\mathbf{y}_n)) \frac{\partial y_{nk}}{\partial\omega_{\mu}} \frac{\partial y_{nj}}{\partial\omega_{\nu}} \Big|_{\omega=\omega^*} \\
&= \sum_{nkj} \sigma_k(\mathbf{y}_n) (\delta_{kj} - \sigma_j(\mathbf{y}_n)) \frac{\partial y_{nk}}{\partial\omega_{\mu}} \frac{\partial y_{nj}}{\partial\omega_{\nu}} \Big|_{\omega=\omega^*}.
\end{aligned}$$

To express the Hessian in a similar way, we denote the target vector of a sample x_n as t_{ni} . Then:

$$\begin{aligned}
\frac{\partial^2}{\partial\omega\partial\omega} L &= - \frac{\partial^2}{\partial\omega\partial\omega} \sum_c \int P_{\omega^*}(c|x) P(x) \log [P_{\omega}(c|x) P(x)] dx \\
&\sim - \frac{\partial^2}{\partial\omega\partial\omega} \sum_{ni} t_{ni} \log [P_{\omega}(i|x_n) P(x_n)] \\
&= - \frac{\partial^2}{\partial\omega\partial\omega} \sum_{ni} t_{ni} \log \sigma_i(\mathbf{y}_n) \\
&= - \frac{\partial}{\partial\omega} \sum_{nik} t_{ni} [\delta_{ik} - \sigma_k(\mathbf{y}_n)] \frac{\partial y_{nk}}{\partial\omega} \\
&= \sum_{nijk} t_{ni} \sigma_k(\mathbf{y}_n) (\delta_{kj} - \sigma_j(\mathbf{y}_n)) \frac{\partial y_{nk}}{\partial\omega} \frac{\partial y_{nj}}{\partial\omega} - \sum_{nik} t_{ni} [\delta_{ik} - \sigma_i(\mathbf{y}_n)] \frac{\partial^2 y_{nk}}{\partial\omega\partial\omega} \\
&= \sum_{njk} \sigma_k(\mathbf{y}_n) (\delta_{kj} - \sigma_j(\mathbf{y}_n)) \frac{\partial y_{nk}}{\partial\omega} \frac{\partial y_{nj}}{\partial\omega} - \sum_{nik} t_{ni} [\delta_{ik} - \sigma_i(\mathbf{y}_n)] \frac{\partial^2 y_{nk}}{\partial\omega\partial\omega} \\
&= g(\omega) - \sum_{nik} t_{ni} [\delta_{ik} - \sigma_i(\mathbf{y}_n)] \frac{\partial^2 y_{nk}}{\partial\omega\partial\omega}. \quad (21)
\end{aligned}$$

Algorithm 1 Orthogonalized columns in GN matrix

Input: Model f , Adaptation algorithm $\mathcal{A}$
 Training taskset $D = \{(\mathcal{D}^{(i)\text{sup}}, \mathcal{D}^{(i)\text{val}})\}_{i=1}^M$
 Trained meta-parameters $\hat{\omega}$
 # output columns N_{orth}
 vector buffer size N_{max}

Output: Matrix with orthogonal columns V_{orth}

```

1:  $V_{\text{orth}} \leftarrow$  empty matrix
2: for all  $(\mathcal{D}^{\text{sup}}, \mathcal{D}^{\text{val}}) \leftarrow D$  do
3:    $\hat{\theta} \leftarrow \mathcal{A}(\mathcal{D}^{\text{sup}}, \hat{\omega})$ 
4:   for all  $(x, t) \leftarrow \mathcal{D}^{\text{val}}$  do
5:      $\mathbf{y} \leftarrow f(x; \hat{\theta})$ ,  $\boldsymbol{\sigma} \leftarrow \text{Softmax}(\mathbf{y})$ 
6:      $\mathbf{v} \leftarrow \partial \mathbf{y} / \partial \omega|_{\omega=\hat{\omega}}$ 
7:      $\mathbf{v} \leftarrow \text{Sqrt}(\text{DiagonalMatrix}(\boldsymbol{\sigma}) - \boldsymbol{\sigma} \boldsymbol{\sigma}^T) \cdot \mathbf{v}$ 
8:      $V_{\text{orth}} \leftarrow \text{HorizontalStack}(V_{\text{orth}}, \mathbf{v})$ 
9:   end for
10:  if # columns of  $V_{\text{orth}} > N_{\text{max}}$  then
11:     $V_{\text{orth}} \leftarrow \text{OrthogonalColumns}(V_{\text{orth}}, N_{\text{orth}})$ 
12:  end if
13: end for
14:  $V_{\text{orth}} \leftarrow \text{OrthogonalColumns}(V_{\text{orth}}, N_{\text{orth}})$ 
15: return  $V_{\text{orth}}$ 

```

Algorithm 2 Orghogonal Columns

Input: Matrix V , # output columns N_{orth}

Output:
 Matrix with orthogonal columns V_{orth}

```

1: Compute orthogonal matrix  $O$ , diagonal matrix  $\Lambda$ , such that  $V^T V = O \Lambda O^T$ 
2: List of eigenvalues  $\boldsymbol{\lambda} \leftarrow \text{DiagonalPart}(\Lambda)$ 
3: Extract a set of indices  $L = \{i_1, \dots, i_{N_{\text{orth}}}\}$  such that  $\boldsymbol{\lambda}[i_1], \dots, \boldsymbol{\lambda}[i_{N_{\text{orth}}}]$  is the largest  $N_{\text{orth}}$  eigenvalues
4:  $V_{\text{orth}} \leftarrow V O[:, L]$ 
5: return  $V_{\text{orth}}$ 

```

Algorithm 3 Orthogonal Columns in pseudo-inverse GN matrix

Input: Matrix with orthogonal columns V_{orth} of size $n \times m$

Output:
 Matrix with orthogonal columns V_{orth}^+

```

1:  $V_{\text{orth}}^+ \leftarrow$  empty matrix
2: for all  $j \leftarrow [1, \dots, m]$  do
3:    $\mathbf{v} \leftarrow V_{\text{orth}}[:, j]$ 
4:   if  $|\mathbf{v}| > 0$  then
5:      $\mathbf{v} \leftarrow \mathbf{v} / |\mathbf{v}|^2$ 
6:   end if
7:    $V_{\text{orth}}^+ \leftarrow \text{HorizontalStack}(V_{\text{orth}}^+, \mathbf{v})$ 
8: end for
9: return  $V_{\text{orth}}^+$ 

```

The second-to-last equation proves Eq. 10. From the last equation, we see that if the distributions $P(X|\omega^*)$ and $P(X|\omega)$ are accurately approximated by the training data samples and the model’s outputs, respectively, the first term in Eq. 10 becomes equivalent to the Fisher information metric evaluated at ω . By comparing Eq. 20 and Eq. 21, we see that if ω is near to ω^* , the second term of Eq. 21 drops, and the Hessian is well approximated by the Fisher information metric evaluated at ω , that is, $g_{\mu\nu}(\omega)$.

A.6 Implementation

We comment on the implementation for computing the approximated Hessian Eq. 11 and its pseudo-inverse H^+ in Eq. 12. Here we restate the GN-matrix approximation of the Hessian:

$$H \sim \sum_{nj} (\mathbf{V})_{\mu(nj)} (\mathbf{V}^T)_{(nj)\nu} = V V^T \quad (22)$$

where we define the matrix V as the horizontal concatenation of q dimensional column vectors $\mathbf{V}_{(nj)}$. The block specified by n , associated with each query in each training task, satisfies

$$\sum_j (\mathbf{V})_{\mu(nj)} (\mathbf{V}^T)_{(nj)\nu} = \sum_{jk} \sigma_k(\mathbf{y}_n) (\delta_{kj} - \sigma_j(\mathbf{y}_n)) \frac{\partial y_{nk}}{\partial \omega} \frac{\partial y_{nj}}{\partial \omega},$$

where μ, ν are the meta-parameter indices, n is the training task and query index, and k, j are the output tensor indices. Although the Gauss-Newton matrix approximation avoids the $\mathcal{O}(pq^2)$ computational cost for handling a network with

p model parameters and q meta-parameters, the large size of H , i.e. $q \times q$, leads to a large storage/memory cost. We can mitigate it by keeping only V and conducting all manipulation without evaluating the matrix product in Eq. 22. However, when the training taskset is large, the number of columns of V itself is also large.³ To resolve this issue, we orthogonalize the columns to remove the redundancy of linear dependency among them. As explained below, this can be done by sequentially orthogonalizing column vectors (Algorithm 1), thereby avoiding the costs of accumulating all terms and orthogonalizing a large set of vectors at once. We also note that, in conducting the experiments in this paper, methods of further approximating the GN-matrix, such as K-FAC (Martens and Grosse, 2015), are not necessary.

First, we show that removing columns from a matrix by the orthogonalization of Algorithm 2 preserves the matrix product.

Proposition 2. Let V and N_{orth} be the input to Algorithm 2, and V_{orth} be its output. If N_{orth} is greater than or equal to the number of linearly independent columns of V , then

$$VV^T = V_{\text{orth}}V_{\text{orth}}^T.$$

Proof. We use the notations in Algorithm 2. Let N be the number of columns of V . From $O^TV^TVO = \text{diag}(\lambda_1, \dots, \lambda_N)$, we see the columns of $VO = [\mathbf{v}_1, \dots, \mathbf{v}_N]$ are orthogonal to each other: $\mathbf{v}_i \cdot \mathbf{v}_j = 0$ for $i \neq j$, and their squared norms are the eigenvalues: $\mathbf{v}_i \cdot \mathbf{v}_i = \lambda_i$. Noting VV^T is positive semi-definite and $N_{\text{orth}} \geq \text{rank}VV^T$, we see that $\lambda_i = 0$ and hence $\mathbf{v}_i = \mathbf{0}$ for $i \notin L$. Thus we obtain

$$VV^T = VOO^TV^T = \sum_{i=0}^N \mathbf{v}_i\mathbf{v}_i^T = \sum_{i \in L} \mathbf{v}_i\mathbf{v}_i^T = V_{\text{orth}}V_{\text{orth}}^T.$$

□

Let us consider a split of columns of a matrix $V = [V', V'']$. A proposition similar to the above also holds in the case that Algorithm 2 is applied to the submatrix V' .

Proposition 3. Let V' and N_{orth} be the input to Algorithm 2, and V'_{orth} be its output. If N_{orth} is greater than or equal to the number of linearly independent columns of V' , then

$$VV^T = V'_{\text{orth}}V'_{\text{orth}}^T + V''V''^T.$$

Proof. Because the number of independent columns of V is greater than or equal to that of V' , N_{orth} is greater than or equal to the latter. Then applying Proposition 2 to V', N_{orth} leads to the claim. □

Owing to Proposition 3, we can orthogonalize the column vectors and drop the zero vectors sequentially during the summation of Eq. 22. More precisely, the following holds.

Proposition 4. We use the notation of Algorithm 1. Let V in Eq. 22 be an input to it and N_{orth} be an integer greater than the number of linearly independent columns. If $N_{\text{orth}} \leq N_{\text{max}}$, then the summation of Eq. 22 is computed by the output V_{orth} of Algorithm 1:

$$VV^T = V_{\text{orth}}V_{\text{orth}}^T.$$

Proof. This is immediately derived from Proposition 3 □

Next, we show that H^+ can also be written as a summation of factorized forms.

Proposition 5. Let V, V_{orth} be the matrices specified in Proposition 4. The pseudo inverse of the GN matrix Eq. 22 can be written in a factorized form and is computed by the output V_{orth}^+ of Algorithm 3:

$$(VV^T)^+ = V_{\text{orth}}^+V_{\text{orth}}^T.$$

³For example, if the network has $q = 100k$ meta-parameters, the training taskset has $M = 1000$ tasks, and each task is a 5-way-5-shot problem ($C = 5$ output size, $Q = 5 \times 5$ queries per task), then the number of matrix elements is $qCQM = 10^5 \times 5 \times 25 \times 1000 \sim 10^{10}$, which makes it challenging to use floating-point numbers of 16, 32, or 64-bit precision.

Proof. From the proof of Proposition 2, we can write VV^T as follows:

$$H = VV^T = \sum_{i=0}^N \mathbf{v}_i \mathbf{v}_i^T \quad \mathbf{v}_i \cdot \mathbf{v}_j = 0 \text{ for } i \neq j$$

Hence, if $|\mathbf{v}_i| > 0$, $\mathbf{v}_i$ is an eigenvector of H with the positive eigenvalue $\lambda = \mathbf{v}_i \cdot \mathbf{v}_i$. Let S be the set of vectors of this kind ($S = \{\mathbf{v}_i | i \in [1, \dots, N], |\mathbf{v}_i| > 0\}$). Because vectors in the orthogonal complement to the subspace spanned by S are eigenvectors of zero eigenvalue, the pseudo-inverse H^+ is uniquely determined by the following conditions:

$$H^+ H \mathbf{w} = \mathbf{w} \text{ if } \mathbf{w} \in S$$

$$H^+ H \mathbf{w} = \mathbf{0} \text{ if } \mathbf{w} \cdot \mathbf{v}_i = 0 \text{ for } \forall \mathbf{v}_i \in S$$

These can be checked explicitly for $V_{\text{orth}}^+ V_{\text{orth}}^{+T}$:

$$(V_{\text{orth}}^+ V_{\text{orth}}^{+T}) (VV^T) \mathbf{w} = \left(\sum_{\mathbf{v} \in S} \frac{1}{|\mathbf{v}|^4} \mathbf{v} \mathbf{v}^T \right) \left(\sum_{\mathbf{v} \in S} \mathbf{v} \mathbf{v}^T \right) \mathbf{w} = \begin{cases} \mathbf{w} & \text{if } \mathbf{w} \in S \\ \mathbf{0} & \text{if } \mathbf{w} \cdot \mathbf{v} = 0 \text{ for } \forall \mathbf{v} \in S \end{cases}$$

Thus, we see that $H^+ = V_{\text{orth}}^+ V_{\text{orth}}^{+T}$ □

We have seen that we can implement the accumulation of the vector product elements in Eq. 22 by preparing a buffer, sequentially adding the column vectors from each training task to it, orthogonalizing the elements inside it, and dropping zero elements (Algorithm 1), and that the pseudo-inverse is given by appropriate rescaling of those vectors (Algorithm 3). Note that the number of independent columns in V must be small for the above method to be feasible. Fortunately, it is expected to be small, and most columns are dropped as zero vectors. This number corresponds to the non-flat directions of the Hessian, representing the constraints imposed by the loss minimization condition, and typically, it is at most the number of training tasks.

In the experiments of Section 5.1.2, and 5.2, in accumulation of the vector product elements, N_{orth} in Algorithm 1 is set to 8192 and at each time when the number of accumulated vectors exceeds $N_{\text{max}} = 2 \times N_{\text{orth}}$, the columns are orthogonalized, the vectors with the largest N_{orth} norms are kept, and the other vectors are dropped.

After that, we calculate the pseudo-inverse of the Hessian using Algorithm 3. To determine the number of vectors treated as non-zero, we calculate the pseudo-inverses under different assumptions on the number of positive eigenvalues, and choose the best one in terms of self-rank. We set the assumptions that the Hessian has 64, 128, 256, 512, 1024, 2048, 4096, and 8192 positive eigenvalues. For each of them, we retain the assumed numbers of eigenvalues in descending order, treat the other elements as zero, compute the self-rank using the proposed influence function (Eq. 12), and choose the assumption that gives the smallest self-rank. If different assumptions provide the same self-rank, the one with the smallest number is chosen.

B Experimental details

B.1 Dataset

In the experiments below, we use the MiniImagenet and Omniglot datasets. Both are commonly used datasets for measuring model performance in the few-shot learning problem. In addition, to fill their shortcomings, we define the following two datasets.

Synthetic dataset. To validate the proposed method with the exact form of Eq. 12, we employ a lightweight network and a taskset that the network can learn easily. We create tasksets of clustered data points in a two-dimensional space using Gaussian distributions. To create each normal task, we first sample cluster centers around the origin of the plane with a standard deviation of 1. Then, the members of each cluster are sampled around their center with a standard deviation of 0.1, and a unique label is assigned to each cluster. We also use noise tasks consisting of data points sampled around the origin with a standard deviation of 1 and assigned random labels.

Fig. 7 shows examples of a normal task and a noise task in the 3-way-5-shot problem.

FC60 dataset. To investigate the relation between influence scores and semantic similarity, we need a dataset in which training and test tasks share semantic properties. We define a new image dataset, the *FC60 dataset*, in which each image is assigned hierarchical labels specifying the super- and subclass of the image. This is achieved by utilizing the FC100 dataset (Oreshkin et al., 2018), which curates few-shot-learning tasks from CIFAR-100 (Krizhevsky, 2009).

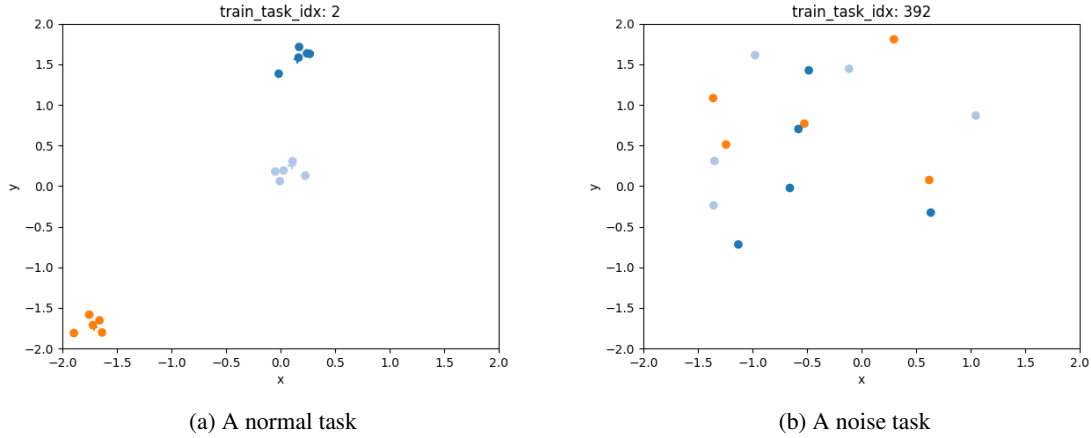


Figure 7: Examples of synthetic tasks generated from Gaussian distributions.

Table 5: Superclasses and subclasses of FC60 dataset

superclass	subclass	
	train	test
fish	[aquarium fish, flatfish, ray]	[shark, trout]
flowers	[orchids, poppies, roses]	[sunflowers, tulips]
food containers	[bottles, bowls, cans]	[cups, plates]
fruit and vegetables	[apples, mushrooms, oranges]	[pears, sweet peppers]
household electrical devices	[clock, keyboard, lamp]	[telephone, television]
household furniture	[bed, chair, couch]	[table, wardrobe]
large man-made outdoor things	[bridge, castle, house]	[road, skyscraper]
large natural outdoor scenes	[cloud, forest, mountain]	[plain, sea]
reptiles	[crocodile, dinosaur, lizard]	[snake, turtle]
trees	[maple, oak, palm]	[pine, willow]
vehicles 1	[bicycle, bus, motorcycle]	[pickup truck, train]
vehicles 2	[lawn-mower, rocket, streetcar]	[tank, tractor]

The tasks in FC100 are grouped based on the superclasses defined in CIFAR100, and the train, validation, and test splits are constructed in a way that no superclass appears in more than one split. We split the FC100 training taskset by dividing the subclasses of each of its 12 superclasses into two sets of subclasses for new training and test tasksets. This results in train/test splits that have 60 subclasses in total and share 12 superclasses (see Table 5 for details). The implementation is based on the FC100 dataset provided in learn2learn and is achieved by applying filters to the subclasses at each data acquisition.

Table 5 shows the superclasses and subclasses used in FC60. Fig. 8 shows examples of a test task and a training task.

B.2 Distinction of Tasks

Setup We employ MAML as the learning algorithm and train a two-layer fully connected network with widths of 32 and 5, using ReLU activations and batch normalization layers (1,285 parameters) with 1000 meta-batches sampled from 128 training tasks in each dataset. For the MiniImagenet dataset, the model’s input is a 32-dimensional feature vector extracted from the image using the Bag-of-Visual-Words (Csurka et al., 2002) with SIFT descriptors (Lowe, 1999, 2004) and k-means clustering. For the Omniglot dataset, it is a 36-dimensional feature vector extracted from the image by applying 2-

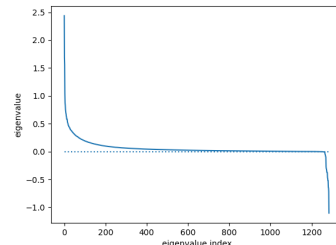


Figure 9: Eigenvalues of the Hessian before the pruning in 5.1.1

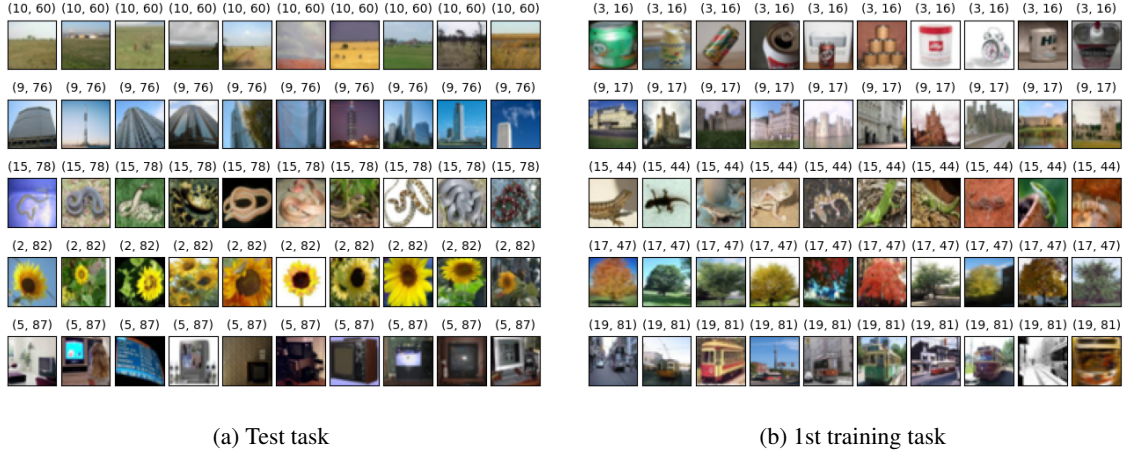


Figure 8: Examples of FC60 tasks. Left: A test task. Right: The training task ranked 1st by TLXML in terms of the influence on the left test task. The pair of labels on each image represents the semantic labels (superclass, subclass) obtained from CIFAR-100. The superclass 9 (large man-made outdoor things) and 15 (reptiles) are shared by them.

Table 6: Effect of Hessian pruning(MiniImagenet). “# eigenvalues” denotes the number of eigenvalues treated as positive in computing the pseudo-inverse (the 1st row is the original Hessian). The 2nd row corresponds to the pruned Hessian, where all 92 negative eigenvalues are set to zero. The 2nd column shows the self-ranks in the tests without degradation. and the remaining columns show correlation coefficients between the two degradation parameters and the self-ranks or self-scores. Values are reported as means and standard deviations across the 128 tasks. $\pm$ means the average and the standard deviation across 128 test tasks.

# eigenvalues	selfrank(avg $\pm$ std)	correlation with degradation(avg $\pm$ std)			
		alpha/rank	alpha/score	ratio/rank	ratio/score
1285	12.6 $\pm$ 18.9	0.51 $\pm$ 0.32	-0.41 $\pm$ 0.29	0.36 $\pm$ 0.32	-0.11 $\pm$ 0.31
1193	0.0 $\pm$ 0.0	0.69 $\pm$ 0.21	-0.69 $\pm$ 0.22	0.46 $\pm$ 0.30	-0.15 $\pm$ 0.34
512	0.0 $\pm$ 0.0	0.71 $\pm$ 0.12	-0.96 $\pm$ 0.06	0.63 $\pm$ 0.09	-0.89 $\pm$ 0.13
256	0.0 $\pm$ 0.0	0.71 $\pm$ 0.11	-0.95 $\pm$ 0.08	0.62 $\pm$ 0.11	-0.88 $\pm$ 0.14
128	0.0 $\pm$ 0.0	0.72 $\pm$ 0.10	-0.94 $\pm$ 0.04	0.63 $\pm$ 0.10	-0.86 $\pm$ 0.15
64	0.0 $\pm$ 0.0	0.71 $\pm$ 0.12	-0.92 $\pm$ 0.06	0.66 $\pm$ 0.12	-0.77 $\pm$ 0.20
32	0.0 $\pm$ 0.2	0.72 $\pm$ 0.16	-0.85 $\pm$ 0.12	0.68 $\pm$ 0.14	-0.68 $\pm$ 0.22
16	2.0 $\pm$ 3.2	0.67 $\pm$ 0.22	-0.69 $\pm$ 0.20	0.61 $\pm$ 0.22	-0.46 $\pm$ 0.27
8	8.6 $\pm$ 9.1	0.55 $\pm$ 0.26	-0.53 $\pm$ 0.23	0.47 $\pm$ 0.27	-0.29 $\pm$ 0.31

dimensional FFT and clipping the 6×6 image at the center.

MiniImagenet In the experiments in Section 5.1.1

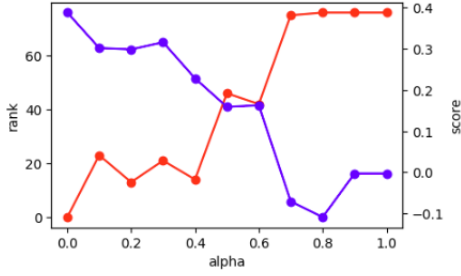
with MiniImagenet, we encounter negative eigenvalues of the Hessian. We provide some details here. Figure 9 shows 1285 eigenvalues arranged from the largest to the smallest. We observe that large positive eigenvalues mainly appear in the first several hundred elements, and also that negative eigenvalues appear in the tail.

In addition to the results mentioned in the main paper, as a sanity check, we checked that reducing the similarity of training and test tasks leads to a degradation of the self-ranks. We degrade the test tasks by darkening their images and examine whether the ranks and scores of the originally identical training tasks (referred to as *self-ranks* and *self-scores*) worsen as the similarity decreases.

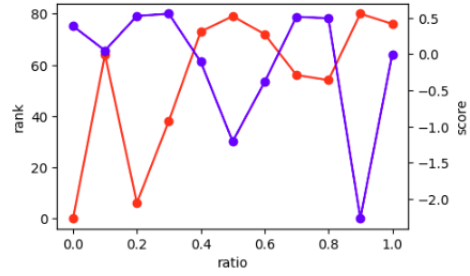
Figure 10 shows examples of the results obtained with MiniImagenet. We observe that the ranks and scores tend to get worse as the darkness (parameterized by α) or the portion (parametrized by *ratio*) of dark images in the test task increases. Those are example plots in the case of a small amount of Hessian pruning. We investigate the effects of pruning on the adequacy of the influence scores by looking at the correlations between the degradation parameters (α and *ratio*) and the two influence measures(self-ranks and self-scores), which are shown in the third to sixth columns of Table 6. We observe that appropriate pruning improves those correlation values.

Table 7: Effect of pruning the Hessian(Omniglot). See table 6 for notations. The second row is the case that we only prune the 428 negative eigenvalues. There are cases in which increasing α does not change the self-ranks. The numbers of those cases are shown in the brackets, and they are removed from the statistics because the correlation coefficients are not defined for them.

# eigenvalues	selfrank(avg $\pm$ std)	correlation with degradation(avg $\pm$ std)			
		alpha/rank	alpha/score	ratio/rank	ratio/score
1413	66.0 $\pm$ 36.9	0.15 $\pm$ 0.48(21)	0.06 $\pm$ 0.50	0.02 $\pm$ 0.34	0.03 $\pm$ 0.23
985	7.8 $\pm$ 10.0	0.48 $\pm$ 0.12(2)	-0.37 $\pm$ 0.33	0.25 $\pm$ 0.28	0.01 $\pm$ 0.23
512	0.8 $\pm$ 5.0	0.50 $\pm$ 0.00(1)	-0.50 $\pm$ 0.00	0.35 $\pm$ 0.26	-0.15 $\pm$ 0.23
256	1.6 $\pm$ 6.3	0.50 $\pm$ 0.00(1)	-0.50 $\pm$ 0.00	0.34 $\pm$ 0.27	-0.12 $\pm$ 0.25
128	2.9 $\pm$ 7.4	0.50 $\pm$ 0.00(1)	-0.50 $\pm$ 0.00	0.36 $\pm$ 0.27	-0.11 $\pm$ 0.24
64	4.3 $\pm$ 8.0	0.50 $\pm$ 0.00(1)	-0.50 $\pm$ 0.00	0.36 $\pm$ 0.26	-0.13 $\pm$ 0.23
32	6.4 $\pm$ 8.8	0.50 $\pm$ 0.00(1)	-0.49 $\pm$ 0.09	0.33 $\pm$ 0.26	-0.08 $\pm$ 0.24
16	8.4 $\pm$ 10.0	0.50 $\pm$ 0.00(1)	-0.50 $\pm$ 0.00	0.32 $\pm$ 0.30	-0.08 $\pm$ 0.25
8	11.8 $\pm$ 12.5	0.50 $\pm$ 0.00(4)	-0.50 $\pm$ 0.00	0.29 $\pm$ 0.29	-0.06 $\pm$ 0.24



(a) Example of the effect of increasing α . $\alpha=0$ means the test task without degradation. $\alpha=1$ means all the images in the test task are completely black.



(b) Example of the effect of increasing the ratio. ratio=0 means the test task without degradation. ratio=1 means all the images in the test task are dark.

Figure 10: Test with degraded training tasks(MiniImagenet). The parameters α and *ratio* specify the darkness of images, and the number of dark images in each degraded task, respectively. The red and blue lines represent ranks and scores, respectively. Both examples were performed with the Hessian pruned to retain only the 1193 most significant eigenvalues.

Omniglot We also conduct experiments with the Omniglot dataset. In this case, we encounter 428 negative eigenvalues of the Hessian. The effects of pruning on the self-rank are shown in the first column of Table 7. Tests with the degradation of those test tasks are also conducted. Again, we observe that the correlations are improved to some extent by pruning. However, those correlations are weaker than in the cases of MiniImagenet (Table 6). A possible reason for the weak correlations is that the tasks in Omniglot are easier to adapt to than those in MiniImagenet, which makes the trained model less sensitive to degradation.

B.3 Distinction of Normal and Noise Task Distributions

Setup. For the networks used in the experiments below, we employ ReLu activation and batch normalization.

Synthetic dataset To examine the method with the exact form of Eq. 4, we employ a lightweight network and easy tasksets that can be learn easily. Using 1024 training tasks of the 3-ways-5-shots problem, including 128 noise tasks, in the synthetic dataset described in B.1, we train a 3-layer fully connected network with 2-dimensional input, 4-4 hidden dimensions, 3-dimensional output layers (63 parameters) with 30,000 meta-batches, and then calculated I^{meta} . To determine the number of positive eigenvalues of an exact Hessian, the average self-ranks over the training tasks were calculated with different assumptions on the number of positive eigenvalues(4, 8, 16, 32, and 63). 32 is chosen as the one giving the best average self-rank. The result is described in the main paper.

Additionally, we calculate the scores using an approximated Hessian. To obtain the Hessian, we accumulate vector products across the training tasks with a buffer size of 32.⁴ We obtain a result similar to that for the exact Hessian and observed that 124 of 128 tests resulted in the proper order of the training task distributions in terms of their mean values. This count exceeds the average count (= 64) of the binomial distribution by 10.6σ under the hypothesis of random ordering. We also check the correlations between the scores calculated with the exact and approximated

⁴In terms of the parameters in Algorithm 1, $N_{\text{orth}} = 32$, $N_{\text{max}} = 2 \times 32$

Table 8: Experiments with a CNN trained by MAML(MiniImagenet dataset, combined with noise image tasks). 128 test tasks were selected from the test taskset of pure MiniImagenet. The numbers of proper tests are shown in the second-to-last column for each training setting. The standard deviation σ under the null-hypothesis of random ordering is $\sqrt{128 \times 0.5^2} \sim 5.66$. $\pm$ means the average and standard deviation across 5 runs of training.

learning method	dataset	training tasks		accuracy test	train	proper tests	$[\sigma]$
		total	noise			[count]	
MAML	MI	128	16	0.314 ± 0.003	1.000 ± 0.000	16.4 ± 12.9	$-8.4 \pm 2.3 \sigma$
MAML	MI	256	32	0.323 ± 0.014	1.000 ± 0.000	14.4 ± 7.6	$-8.8 \pm 1.3 \sigma$
MAML	MI	512	64	0.345 ± 0.006	0.995 ± 0.005	51.2 ± 16.5	$-2.3 \pm 2.9 \sigma$
MAML	MI	1024	128	0.368 ± 0.007	0.902 ± 0.049	99.6 ± 20.5	$6.3 \pm 3.6 \sigma$
MAML	MI	2048	256	0.435 ± 0.006	0.828 ± 0.014	73.4 ± 6.0	$1.7 \pm 1.1 \sigma$
MAML	MI	4096	512	0.520 ± 0.012	0.762 ± 0.008	89.2 ± 10.3	$4.5 \pm 1.8 \sigma$
MAML	MI	8192	1024	0.552 ± 0.012	0.717 ± 0.007	85.6 ± 11.1	$3.8 \pm 2.0 \sigma$

Hessians. The Pearson correlation of the influence scores of the 8192 training tasks calculated for each of the 128 tests is 0.715 ± 0.092 .

MiniImagenet and Omniglot. We train 3-conv $\times$ 32-filter + 1-fully-connected layers networks (21,029 parameters for MiniImagenet, 19,178 parameters for Omniglot) using MAML and 4-conv $\times$ 32-filter feature extractors (28,896 parameters for MiniImagenet, 28,320 parameters for Omniglot) using Protonet⁵. We use 128, 256, 512, 1024, 2048, 4096, and 8192 training tasks for each dataset, and combined them with 16, 32, 64, 128, 256, 512, and 1024 noise tasks, respectively. We create noise tasks by replacing the image tensors in each training task with uniform noise tensors of the same shape(noise images). In MAML training, the networks are trained with 80,000 meta-batches and 40,000 meta-batches for MiniImagenet and Omniglot, respectively. In Protonet training, the networks are trained with 20,000 meta-batches and 10,000 meta-batches for MiniImagenet and Omniglot, respectively. We conduct 5 runs of training with seeds 0, 1, 2, 3, and 42. We evaluate the influence scores of the training tasks on 128 test tasks, and counted the number of proper tests based on Eq. 7, and 9 with the projected influence on the meta-parameters (Eq. 12) and the Gauss-Newton matrix approximation of the Hessian (Eq. 11). When we calculate an approximated Hessian (Eq. 11) after each training, we accumulate vector products over the training tasks with the buffer size set to 8192⁶, and then truncate them before evaluating the influence scores. To determine the truncated number, we calculate the average self-rank over the training tasks with different assumptions on the number of positive eigenvalues (64, 128, 256, 512, 1024, 2048, 4096, and 8192). We choose the most appropriate assumption that gives the best self-rank. When different assumptions provide the same self-rank (like 0 ± 0), we choose the assumption of the smallest number.

Table 8, 9, 10, and 11 show the results of the experiments. We observe that when the number of training tasks is sufficiently large, the number of proper tests rejects the hypothesis of random ordering, suggesting that our scoring method satisfies property 2 statistically. Noting that the statistical significance is weak in the results of a small number of training tasks, we can see that Property 2 arises as an effect of generalization. Fig.11 demonstrates this as the relation between the test accuracy and the number of proper tests in the case of MAML and MiniImagenet. In that case, in the region of 128 and 256 training tasks, the number of proper tests is exceeded by the number of the opposite

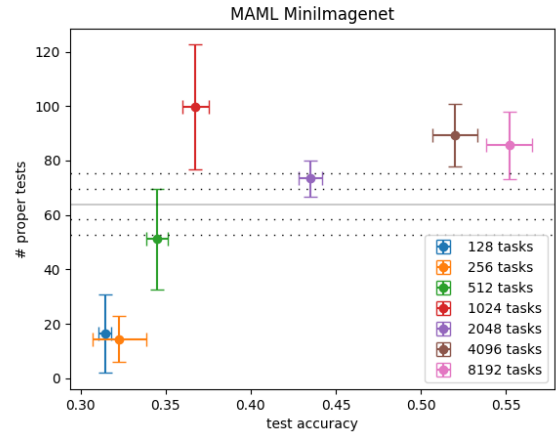


Figure 11: Effect of generalization with a CNN trained by MAML(MiniImagenet dataset, combined with noise image tasks). The solid horizontal line at $y=64$ represents the mean value under the hypothesis of random ordering. The dashed horizontal lines represent the mean value $\pm 1\sigma$ and $\pm 2\sigma$ under the hypothesis of random ordering.

⁵We choose networks such that the number of model parameters is around $\sim 20,000$, and the test accuracy does not decrease continuously during the training with 8192 tasks from the dataset in each setting.

⁶In terms of the parameters in Algorithm 1, $N_{\text{orth}} = 8192$, $N_{\text{max}} = 2 \times 8192$

Table 9: Experiments with a CNN trained by MAML(Omniglot dataset, combined with noise image tasks). See Table 8 for notations.

learning method	dataset	training tasks		accuracy test	train	proper tests	
		total	noise			[count]	$[\sigma]$
MAML	OM	128	16	0.668 ± 0.017	1.000 ± 0.000	68.6 ± 6.0	$0.8 \pm 1.1 \sigma$
MAML	OM	256	32	0.759 ± 0.009	1.000 ± 0.000	77.0 ± 7.9	$2.3 \pm 1.4 \sigma$
MAML	OM	512	64	0.824 ± 0.010	0.984 ± 0.032	118.4 ± 4.4	$9.6 \pm 0.8 \sigma$
MAML	OM	1024	128	0.882 ± 0.032	0.999 ± 0.003	114.8 ± 7.2	$9.0 \pm 1.3 \sigma$
MAML	OM	2048	256	0.892 ± 0.005	0.987 ± 0.004	90.0 ± 3.9	$4.6 \pm 0.7 \sigma$
MAML	OM	4096	512	0.933 ± 0.006	0.940 ± 0.005	82.2 ± 5.9	$3.2 \pm 1.0 \sigma$
MAML	OM	8192	1024	0.960 ± 0.001	0.906 ± 0.003	94.0 ± 6.0	$5.3 \pm 1.1 \sigma$

Table 10: Experiments with a CNN trained by Prototypical Network(MiniImagenet dataset, combined with noise image tasks). See Table 8 for notations.

learning method	dataset	training tasks		accuracy test	train	proper tests	
		total	noise			[count]	$[\sigma]$
Protonet	MI	128	16	0.397 ± 0.016	0.997 ± 0.001	63.0 ± 15.5	$-0.2 \pm 2.7 \sigma$
Protonet	MI	256	32	0.435 ± 0.018	0.778 ± 0.010	108.2 ± 9.2	$7.8 \pm 1.6 \sigma$
Protonet	MI	512	64	0.496 ± 0.009	0.608 ± 0.037	94.4 ± 8.4	$5.4 \pm 1.5 \sigma$
Protonet	MI	1024	128	0.503 ± 0.015	0.570 ± 0.007	81.8 ± 7.5	$3.1 \pm 1.3 \sigma$
Protonet	MI	2048	256	0.498 ± 0.007	0.550 ± 0.008	84.2 ± 7.6	$3.6 \pm 1.3 \sigma$
Protonet	MI	4096	512	0.497 ± 0.016	0.537 ± 0.007	96.4 ± 9.9	$5.7 \pm 1.7 \sigma$
Protonet	MI	8192	1024	0.509 ± 0.007	0.535 ± 0.008	95.6 ± 5.3	$5.6 \pm 0.9 \sigma$

ones. This can be interpreted as reflecting the tendency that when overfitting occurs, only a few training tasks similar to the test task provide helpful information, and most of the regular training tasks become detrimental.

B.4 Consistency with Semantics

Using 8192 training tasks in the FC60 dataset, which is described B.1, we train 3-conv $\times$ 16-filter + 1-fully-connected layers networks (6469 parameters) with 40,000 meta-batches using MAML and 4-conv $\times$ 32-filter feature extractors (28,896 parameters) with 40,000 meta-batches using Protonet.

Table 12 presents the complete list of results from counting the proper tests based on the number of superclasses shared between the training and test tasks. This demonstrates that TLXML effectively identifies subpopulations in the training taskset that share semantic properties with the test tasks, distinguishing them from other training tasks.

In addition, we consider classifying the labels in a test task and defining the training subpopulation for each of the classified label groups. Table 13, and 14 show the results from counting the proper tests when the training subpopulation is defined based on the number of shared superclasses with the test labels classified by their recall values achieved in the tests. Although the classification results in fewer statistics for each row, the Tables again demonstrate that TLXML identifies subpopulations in the training taskset that share semantic properties with test tasks. Furthermore, we can observe a tendency for labels with low recall values to define subpopulations with comparatively small significance, implying a correlation between recall and influence scores (i.e., when a label has a good recall, training tasks with labels in the same superclass are scored highly).

B.5 One-Step Update Using TLXML

We examine the effect of the one-step update Eq. 13 from the convergence point of meta-learning. We train a 3-conv $\times$ 16-filter network(6469 parameters), using 8192 training tasks from the FC60 dataset and employing MAML with 40,000 meta-batches. We conduct 5 runs of training with seeds 0, 1, 2, 3, and 42. Each training task is scored using the influence function for the average test loss across 1024 test tasks. For each setting on the number of blocked or enhanced tasks, those tasks are determined by selecting the worst or best training tasks from the scoring results.

Table 15 shows the effects of both blocking and enhancing training tasks, and the impact on the test accuracies in two cases: test datasets with and without shared superclasses (FC60 test taskset and FC100 test taskset, respectively). For each combination of blocked/enhanced and the two test tasksets, we observe cases of improved test accuracy. The

Table 11: Experiments with a CNN trained by Prototypical Network(Omniglot dataset, combined with noise image tasks). See Table 8 for notations.

learning method	dataset	training tasks		accuracy test	train	proper tests [count]	[σ]
		total	noise				
Protonet	OM	128	16	0.937 ± 0.007	1.000 ± 0.000	77.4 ± 4.7	$2.4 \pm 0.8 \sigma$
Protonet	OM	256	32	0.958 ± 0.003	1.000 ± 0.000	61.0 ± 3.4	$-0.5 \pm 0.6 \sigma$
Protonet	OM	512	64	0.963 ± 0.004	0.986 ± 0.004	112.6 ± 3.9	$8.6 \pm 0.7 \sigma$
Protonet	OM	1024	128	0.973 ± 0.002	0.928 ± 0.002	84.0 ± 4.1	$3.5 \pm 0.7 \sigma$
Protonet	OM	2048	256	0.978 ± 0.002	0.901 ± 0.001	81.0 ± 4.1	$3.0 \pm 0.7 \sigma$
Protonet	OM	4096	512	0.978 ± 0.003	0.891 ± 0.001	84.0 ± 4.6	$3.5 \pm 0.8 \sigma$
Protonet	OM	8192	1024	0.979 ± 0.001	0.884 ± 0.008	81.2 ± 5.5	$3.0 \pm 1.0 \sigma$

Table 12: Experiments of distinguishing a training task subpopulation with superclasses shared with the test task from other training tasks. The 2nd column specifies the least number of superclasses shared with each test task, which defines the training tasks 'similar' to the test task. The 4th column shows the number of training tasks satisfying the condition. The 6th column shows the counts of proper tests and their statistical significance. $\pm$ means the average and standard deviations across 1024 test tasks.

learning method	filter label overlap	train tasks		test tasks		σ
		total	filtered	total	proper	
MAML	1	8192	7522 ± 314	1024	618 (+6.6 σ)	16
MAML	2	8192	4781 ± 718	1024	676 (+10.3 σ)	16
MAML	3	8192	1508 ± 422	1024	688 (+11.0 σ)	16
MAML	4	8192	171 ± 69	1024	637 (+7.8 σ)	16
Protonet	1	8192	7522 ± 314	1024	658 (+9.1 σ)	16
Protonet	2	8192	4781 ± 718	1024	672 (+10.0 σ)	16
Protonet	3	8192	1508 ± 422	1024	703 (+11.9 σ)	16
Protonet	4	8192	171 ± 69	1024	644 (+8.3 σ)	16

improvement is slight when the absolute value of the shift parameter ξ is small and there are few blocked/enhanced tasks, as that is close to doing nothing. The improvement is also slight in the region of a large absolute value of the shift parameter ξ and a large number of blocked/enhanced tasks, which is considered to be caused by the deterioration of the linear approximation by the influence function I^{meta} .

The table also shows the results of leave-out retraining. For each set of blocked tasks, the network was trained from scratch with those tasks removed from the dataset. We observe that the leave-out trainings fail to yield a statistically significant gain.

B.6 Computational Resources

The experiments in this paper are carried out in multiple computing environments. In a typical environment, the machine is equipped with an Intel Core i7-7567U CPU with a 3.50GHz clock, 32GB RAM, and an NVIDIA GeForce RTX 2070 GPU, and the operating system is Ubuntu 24.04 LTS.

Table 13: Experiments of distinguishing a training task subpopulation with superclasses shared with the test task (classified by recall values) from other training tasks(MAML). The 2nd column specifies the recall value of each label in the test tasks, and the 3rd column specifies the least number of labels with that recall value. The 6th column shows the number of test tasks whose labels satisfy the conditions. The 5th column shows the number of training tasks 'similar' to the test task, meaning that the superlabels in each of them include those used for conditioning the test task. The 7th column shows the counts of proper tests and their statistical significance. $\pm$ means the average and standard deviations across the test tasks specified by the conditions.

learning method	filter recall	label overlap	train tasks		test tasks		σ
			total	filtered	total	proper	
MAML	1.0	1	8192	2013 $\pm$ 1145	828	440 (+1.8 σ)	14.4
MAML	1.0	2	8192	785 $\pm$ 325	377	205 (+1.7 σ)	9.7
MAML	1.0	3	8192	245 $\pm$ 53	97	57 (+1.7 σ)	4.9
MAML	0.8	1	8192	1833 $\pm$ 1184	819	488 (+5.5 σ)	14.3
MAML	0.8	2	8192	728 $\pm$ 358	427	255 (+4.0 σ)	10.3
MAML	0.8	3	8192	221 $\pm$ 81	136	90 (+3.8 σ)	5.8
MAML	0.8	4	8192	43 $\pm$ 17	20	17 (+3.1 σ)	2.2
MAML	0.6	1	8192	2187 $\pm$ 1105	706	431 (+5.9 σ)	13.3
MAML	0.6	2	8192	793 $\pm$ 319	267	170 (+4.5 σ)	8.2
MAML	0.6	3	8192	217 $\pm$ 81	55	37 (+2.6 σ)	3.7
MAML	0.4	1	8192	2485 $\pm$ 960	549	319 (+3.8 σ)	11.7
MAML	0.4	2	8192	266 $\pm$ 848	138	87 (+3.1 σ)	5.9
MAML	0.4	3	8192	247 $\pm$ 41	16	11 (+1.5 σ)	2.0
MAML	0.2	1	8192	2707 $\pm$ 764	380	223 (+3.4 σ)	9.7
MAML	0.2	2	8192	906 $\pm$ 226	57	35 (+1.7 σ)	3.8
MAML	0.0	1	8192	2888 $\pm$ 87	220	133 (+3.1 σ)	7.4

Table 14: Experiments of distinguishing a training task subpopulation with superclasses shared with the test task (classified by recall values) from other training tasks(Protonet). See Table 13 for notations

learning method	filter recall	label overlap	train tasks		test tasks		σ
			total	filtered	total	proper	
Protonet	1.0	1	8192	2225 $\pm$ 1095	681	396 (+4.3 σ)	13.0
Protonet	1.0	2	8192	804 $\pm$ 326	248	151 (+3.4 σ)	7.9
Protonet	1.0	3	8192	218 $\pm$ 82	56	37 (+2.4 σ)	3.7
Protonet	0.8	1	8192	1652 $\pm$ 1197	838	479 (+4.1 σ)	14.5
Protonet	0.8	2	8192	679 $\pm$ 378	492	290 (+4.0 σ)	11.1
Protonet	0.8	3	8192	207 $\pm$ 90	184	109 (+2.5 σ)	6.8
Protonet	0.8	4	8192	45 $\pm$ 15	36	25 (+2.3 σ)	3.0
Protonet	0.6	1	8192	1874 $\pm$ 1198	790	478 (+5.9 σ)	14.1
Protonet	0.6	2	8192	697 $\pm$ 373	392	233 (+3.7 σ)	9.9
Protonet	0.6	3	8192	194 $\pm$ 92	127	68 (+0.8 σ)	5.6
Protonet	0.6	4	8192	33 $\pm$ 18	20	12 (+0.9 σ)	2.2
Protonet	0.4	1	8192	2311 $\pm$ 1075	589	341 (+3.8 σ)	12.1
Protonet	0.4	2	8192	763 $\pm$ 344	187	118 (+3.6 σ)	6.8
Protonet	0.4	3	8192	162 $\pm$ 88	37	21 (+0.8 σ)	3.0
Protonet	0.2	1	8192	2763 $\pm$ 718	312	174 (+2.0 σ)	8.8
Protonet	0.2	2	8192	855 $\pm$ 244	38	19 (+0.0 σ)	3.1
Protonet	0.0	1	8192	2999 $\pm$ 243	74	44 (+1.6 σ)	4.3

Table 15: Test accuracies after single TLXML-guided updates to MAML-trained models with blocked and enhanced tasks. $\pm$ means the average and standard deviation across 5 runs of MAML training. Bold values outperform the MAML baseline (shown in the bracket) with a unpaired Welch two-sample t -test, $p < 0.05$. $\dagger$ means the average is below the baseline value.

Op.	Dataset (train→test)	# tasks	leave-out			
block	FC60→FC60 (0.663 $\pm$ 0.004)	128				0.662 $\pm$ 0.006 $\dagger$
		256				0.660 $\pm$ 0.007 $\dagger$
		512				0.663 $\pm$ 0.008
		1024				0.661 $\pm$ 0.006 $\dagger$
		2048				0.666 $\pm$ 0.010
Op.	Dataset (train→test)	# tasks	$\xi = -8$	$\xi = -4$	$\xi = -2$	$\xi = -1$
block	FC60→FC60 (0.663 $\pm$ 0.004)	128	0.670 $\pm$ 0.005	0.667 $\pm$ 0.004	0.665 $\pm$ 0.004	0.664 $\pm$ 0.004
		256	0.672$\pm$0.005	0.670 $\pm$ 0.004	0.666 $\pm$ 0.004	0.665 $\pm$ 0.004
		512	0.675$\pm$0.006	0.672$\pm$0.005	0.669 $\pm$ 0.004	0.666 $\pm$ 0.004
		1024	0.673 $\pm$ 0.008	0.676$\pm$0.005	0.671$\pm$0.004	0.668 $\pm$ 0.004
		2048	0.659 $\pm$ 0.009	0.674$\pm$0.007	0.672$\pm$0.004	0.669 $\pm$ 0.004
	FC60→FC100 (0.429 $\pm$ 0.005)	128	0.439$\pm$0.007	0.436 $\pm$ 0.005	0.433 $\pm$ 0.005	0.431 $\pm$ 0.005
		256	0.443$\pm$0.008	0.438$\pm$0.006	0.435 $\pm$ 0.006	0.432 $\pm$ 0.005
		512	0.448$\pm$0.011	0.443$\pm$0.007	0.438$\pm$0.005	0.434 $\pm$ 0.005
		1024	0.452$\pm$0.011	0.448$\pm$0.009	0.441$\pm$0.007	0.437 $\pm$ 0.006
		2048	0.447 $\pm$ 0.013	0.449$\pm$0.011	0.444$\pm$0.008	0.439 $\pm$ 0.007
Op.	Dataset (train→test)	# tasks	$\xi = 1$	$\xi = 2$	$\xi = 4$	$\xi = 8$
enhance	FC60→FC60 (0.663 $\pm$ 0.004)	128	0.664 $\pm$ 0.004	0.666 $\pm$ 0.005	0.668 $\pm$ 0.005	0.672$\pm$0.005
		256	0.666 $\pm$ 0.005	0.668 $\pm$ 0.005	0.672$\pm$0.005	0.676$\pm$0.005
		512	0.668 $\pm$ 0.005	0.672$\pm$0.005	0.677$\pm$0.006	0.677$\pm$0.007
		1024	0.670$\pm$0.005	0.676$\pm$0.005	0.680$\pm$0.006	0.662 $\pm$ 0.010 $\dagger$
		2048	0.675$\pm$0.005	0.680$\pm$0.006	0.676$\pm$0.008	0.595$\pm$0.026$\dagger$
	FC60→FC100 (0.429 $\pm$ 0.005)	128	0.432 $\pm$ 0.005	0.433 $\pm$ 0.004	0.436 $\pm$ 0.005	0.437 $\pm$ 0.007
		256	0.434 $\pm$ 0.005	0.436 $\pm$ 0.005	0.438 $\pm$ 0.007	0.436 $\pm$ 0.012
		512	0.435 $\pm$ 0.005	0.438 $\pm$ 0.007	0.439 $\pm$ 0.010	0.430 $\pm$ 0.012
		1024	0.438$\pm$0.005	0.441$\pm$0.008	0.437 $\pm$ 0.011	0.413$\pm$0.008$\dagger$
		2048	0.440$\pm$0.007	0.442$\pm$0.009	0.429 $\pm$ 0.010	0.390$\pm$0.020$\dagger$